Sādhanā Vol. , No., , pp.–
DOI
© Indian Academy of Sciences

Neural architecture search via trie-based deduplication and zero-cost proxies, with hardware-aware coverage: An empirical study of LLM-guided warm-start

IMAAD HAJWANE[1,*] and JAYSHREE GHORPADE-AHER[1]

[1]Department of Computer Engineering and Technology, School of Computer Science and Engineering, Dr. Vishwanath Karad MIT World Peace University, Pune 411038, India
e-mail: iamimaad2kk3@gmail.com; jayshree.aher@mitwpu.edu.in

Abstract. Neural architecture search (NAS) is costly since the quality of a candidate network is known only after it has been trained, and many searches spend part of that budget on architectures they have already trained or that were never promising. We study a search loop, ArchTree-RMG (a trie searched by rank-normalised UCB with mutation and a zero-cost gate), that attacks both sources of waste: a prefix trie over architecture tokens guarantees that no architecture is trained twice, and a label-free zero-cost gate, built from three training-free proxies standardised on unlabelled architectures, sends only the top 35% of proposals to real training. Proposals come from rank-normalised upper-confidence-bound (UCB) descent of the trie mixed with tournament mutation, and a large language model (LLM) prior can optionally order unexplored branches. On NAS-Bench-201 the gate removes 59–60% of real evaluations and 53% of GPU-hours at a budget of 200 proposals without loss of test accuracy. When every method receives the same GPU-hour budget (1,128 runs, 14 arms, NAS-Bench-201, NAS-Bench-101 and NATS-Bench SSS, 12 seeds), ArchTree with the gate records no significant loss in 261 Holm-corrected comparisons and lies inside the Nemenyi critical difference of the leading method on every benchmark (tied-best on NAS-Bench-201); the same gate also improves random search, regularised evolution and local search. We show that zero-cost proxies behave largely as training-cost detectors in cell-based spaces (Spearman $\rho = 0.78$ and 0.63 with training time) but not in a width space ($\rho = 0.20$), and derive a closed-form criterion, $r_{PA}r_{PD} - r_{DA}$, that correctly predicted the sign of a cost penalty on all seven benchmark–dataset pairs. The LLM prior reaches near-final accuracy two to three times sooner on CIFAR-10/100 using general architectural knowledge rather than memorised benchmark tables, but does not change final accuracy; the tree structure adds nothing beyond deduplication.

Keywords. Neural architecture search, zero-cost proxies, label-free gating, deduplication, equal-compute evaluation, large language models

1 Introduction

Designing a neural network by hand is a slow loop of proposing a layout, training it, measuring its accuracy and trying again. Neural architecture search (NAS) automates this loop [1, 2, 3], but it does not remove its central cost: the only reliable way to learn how good an architecture is remains to train it, which takes hours of accelerator time per candidate. Early NAS systems used thousands of GPU-days [1, 4]. Weight sharing [5, 6] and tabular benchmarks [7, 8, 9] reduced the cost of research, but the number of full training runs a search strategy needs is still the quantity that decides its practical value.

Two kinds of waste are common in sample-based NAS. The first is *repetition*: stochastic proposers such as bandits or language-model generators frequently return architectures that were already trained, and each repeat costs a full evaluation. In our runs an LLM-driven generator spent 30% of its evaluations on repeats, rising to 55% at the largest budget, and a flat bandit without memory spent 67% on average. The second is *futility*: many proposals are poor and could have been rejected cheaply. Zero-cost proxies [10, 11, 12, 13] score an untrained network in one forward and backward pass and could serve as such a filter, but they are usually studied as rankers on fixed pools, often with weights fitted on labelled architectures whose cost is rarely counted, and they are usually compared at an equal number of evaluations rather than at an equal training budget.

This paper studies a search loop, ArchTree, whose final form ArchTree-RMG combines rank-normalised UCB, mutation and a gate, that targets both kinds of waste and then asks, under a fair compute budget, which of its parts actually matter. A prefix trie over the tokens that encode an architecture makes repetition impossible by construction. A *label-free* zero-cost gate, built from three standard proxies that are standardised on unlabelled architectures, decides which proposals deserve real training. Proposals come from upper-confidence-bound (UCB) descent of the trie, with a rank-normalised reward, mixed with tournament mutation. An LLM prior can optionally order unexplored branches. We evaluate on NAS-Bench-201 [8], NAS-Bench-101 [7], the size space of NATS-Bench [9] and HW-NAS-Bench [14],

*For correspondence

with 12 search seeds, 12 held-out seeds for design validation and nine external baselines.

The contributions are empirical and are stated with their limits:

1. A label-free zero-cost gate inside a deduplicating search that cuts real evaluations by 59–60% and GPU-hours by 53% at equal accuracy on NAS-Bench-201, uses no labelled architectures, and raises the accuracy of every search strategy that uses it (section 5.5).

2. An equal-GPU-hour protocol over three benchmarks (1,128 runs) in which ArchTree with the gate never loses significantly to any of 13 arms and stays inside the critical difference of the leading arm arm everywhere (section 5.5).

3. Evidence that zero-cost proxies act as training-cost detectors in cell-based spaces, though not in a width space, together with a closed-form *gate-cost criterion* that predicts, before any search is run, whether penalising cost in the gate helps (section 5.6).

4. Controlled tests of an LLM prior that separate general architectural knowledge from benchmark memorisation and from a size heuristic, and a negative result for the tree structure itself (sections 5.8 and 5.9).

We make no claim that ArchTree is a new state of the art; local search equipped with the same gate does better on two of the three benchmarks. What carries over to other searches is the gate, the evaluation protocol and the explanation of when cost-aware gating helps.

The evidence comes from 12,289 complete, seed-paired searches organised in stages, each answering one question (Table 3). Sections 2–4 cover related work, the method and the setup, section 5 the results, and sections 6–7 the discussion and conclusions.

2 Related work

Search strategies. Reinforcement learning [1, 5], evolution [4, 15] and Bayesian optimisation [16] are the classical NAS search strategies. On tabular benchmarks, simple methods are strong: random search is a hard baseline [17, 18], regularised evolution (RE) is among the best sample-based methods [15, 7], and local search is competitive or better on NAS-Bench-201 [19]. We include all three, as recommended by a recent survey [20].

Tree search. Monte-Carlo tree search over architecture decisions was used in AlphaX [21] and refined by learning the action space in LaNAS [22]; TNAS [23] organises the NAS-Bench-201 space as a tree of operations. We test this assumption directly by comparing the trie with a flat per-position bandit, with and without deduplication. Hierarchical optimistic search [24, 25, 26] assumes a smoothness we measure.

Zero-cost proxies. Training-free scores such as SynFlow [12], NASWOT [11], SNIP [27] and ZiCo [28] correlate with final accuracy, and Abdelfattah *et al.* [10] used them to warm-start evolution, which is the closest prior work to ours. NAS-Bench-Suite-Zero [13] showed that proxy quality varies strongly across spaces and that simple quantities such as parameter count are competitive. We use proxies as a per-proposal gate inside the search, remove the need for labels, and analyse why and when the gate's bias towards large networks is harmful.

LLM-guided NAS. Large language models have been used as architecture generators [29], as mutation operators for code [30] and inside quality-diversity search [31]. They rarely test for recall of benchmark results. We use an LLM only as a prior over unexplored branches and add three controls: hidden operation names, operations described without benchmark names, and a knowledge-free size prior.

Evaluation practice. Equal-evaluation comparisons hide differences in the cost of each evaluation, and few-seed results are fragile [18, 17]; we therefore compare at equal GPU-hours with the statistics of section 4.

3 Method

Naming. The complete system is called ArchTree-RMG: a prefix trie over architecture tokens (ArchTree) searched with **R**ank-normalised UCB, combined with tournament **M**utation, and filtered by a label-free zero-cost **G**ate. Its reduced forms follow the same convention: ArchTree-base is the original trie with raw-reward UCB ($C = \sqrt{2}$, mean backup) and no gate, and ArchTree-RM is ArchTree-RMG without the gate. Where no suffix is given, ArchTree refers to ArchTree-RMG. The results follow four development stages: **S1** = ArchTree-base; **S2** = S1 with a supervised zero-cost gate (weights fitted on 1,500 labelled architectures); **S3** = ArchTree-RM; **S4** = ArchTree-RMG, the final system.

3.1 *Problem setting*

A search space $\mathcal{A}$ is a finite set of architectures, each written as a token sequence $a = (o_1, \ldots, o_L)$, where o_ℓ is drawn from a position-specific vocabulary $\mathcal{V}_\ell$. On NAS-Bench-201, $L = 6$ edges each take one of five operations, so $|\mathcal{A}| = 5^6 = 15{,}625$. Every architecture has a validation accuracy $A(a)$, a test accuracy $T(a)$ and a training cost $\tau(a)$ (GPU-seconds) recorded by the benchmark. A search proposes candidates one at a time; proposal t either receives a *real evaluation*, which reveals $A(a_t)$ and costs $\tau(a_t)$, or is skipped at negligible cost. Given a budget, the search returns the evaluated architecture with the highest validation accuracy, and we report its test accuracy. Searching on validation and reporting test accuracy follows the NAS-Bench-201 protocol [8] and prevents selection on the reported metric.

3.2 *Trie encoding and deduplication*

Every evaluated architecture is inserted into a prefix trie whose depth equals L; each internal node at depth ℓ represents all architectures that share the first ℓ tokens. Insertion and look-up take $O(L)$ time. Before a proposal is evaluated, the loop checks whether its leaf is already present; if so, the

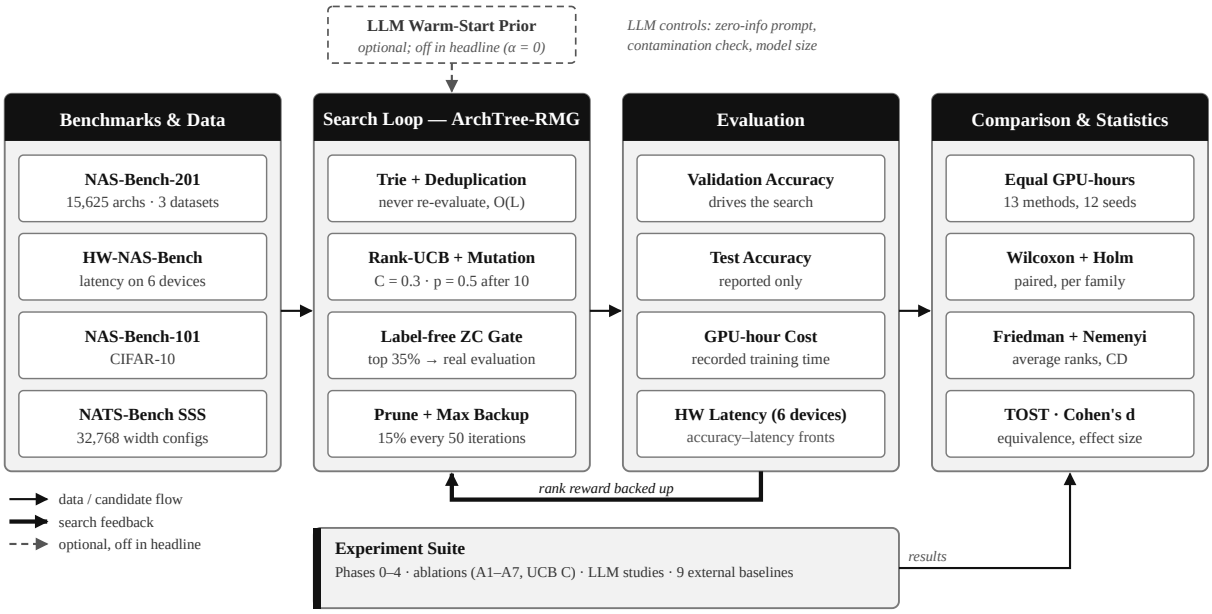


Figure 1. Overview of the study. Benchmarks supply tabulated accuracy, training time and device latency; the ArchTree-RMG loop proposes, deduplicates and gates candidates; validation accuracy drives the search and test accuracy is only reported; all methods are compared at equal GPU-hours with paired and rank-based statistics. The LLM prior (dashed) is switched off in the headline system.

proposal is replaced by a random unvisited architecture. On NAS-Bench-101, tokens are five operations followed by 21 adjacency bits ($L = 26$), and proposals that decode to an invalid or isomorphic cell are redirected in the same way, using the benchmark's graph hash. As a result, no architecture is trained twice, which holds by construction rather than as an empirical property.

3.3 *Rank-normalised UCB with mutation*

At a node p the loop descends to the child

$$c^* = \arg\max_{c \in C(p)} \left[\bar{r}(c) + C \sqrt{\frac{\ln N(p)}{n(c)}} \right], \tag{1}$$

where $N(p)$ and $n(c)$ are visit counts, unvisited children are taken first, and ties are broken at random. The first design, ArchTree-base, used $\bar{r}(c) = A/100$ with $C = \sqrt{2}$. Because NAS-Bench-201 accuracies of good subtrees differ by only a few points, the exploration bonus (about 1–2.5) swamped differences of 0.01–0.05 in the value term and the descent became almost uniform. ArchTree-RMG replaces the value with the percentile rank of the subtree's backed-up accuracy among all accuracies observed in the run,

$$r_t(a) = \frac{1}{t} \sum_{i=1}^{t} \mathbb{1}[A(a_i) \le A(a)] \in [0, 1], \tag{2}$$

uses $C = 0.3$ and backs up the maximum accuracy of each subtree. After ten real evaluations, each proposal is, with probability 0.5, replaced by a mutation of the best of five randomly drawn evaluated architectures (tournament selection as in RE [15]), using the benchmark's own one-token mutation.

3.4 *Label-free zero-cost gate*

Three proxies are computed on an untrained proxy-size network (32 channels, fixed initialisation): the SVD entropy E of the centred final feature map [37], SynFlow S [12], computed in double precision with batch normalisation in evaluation mode, and NASWOT T [11], averaged over three fixed batches of 256 CIFAR-10 images. Each proxy is standardised with the mean and standard deviation of 1,500 randomly drawn *unlabelled* architectures, and the gate score is their unweighted mean,

$$P(a) = \tfrac{1}{3} \left[z(E_a) + z(S_a) + z(T_a) \right]. \tag{3}$$

No accuracy label is used to build P. The first ten proposals always pass; afterwards proposal a_t is evaluated only if

$$P(a_t) \;\ge\; q_{0.65}\{P(a_1), \ldots, P(a_t)\}, \tag{4}$$

that is, if it lies among the highest 35% of the scores observed so far in the run (one proxy evaluation takes 0.28 s). A rejected proposal only increments visit counts on its path. One proxy evaluation takes about 0.28 s on a laptop GPU, roughly 1/5,000 of the mean NAS-Bench-201 CIFAR-10 training time, A supervised variant, with weights fitted to 1,500 labelled architectures, is kept for comparison and its labels are counted as cost.

Algorithm 1 ArchTree-RMG: trie search with rank-normalised UCB, mutation and the label-free zero-cost gate

Input: benchmark $\mathcal{B}$, proposal budget B, gate pass rate $q = 0.35$, warm-up $w = 10$, $C = 0.3$, mutation probability $p_m = 0.5$, prior weight α
Output: test accuracy, real evaluations and GPU-hours of the best-validation architecture

trie $\mathcal{T} \leftarrow \emptyset$; history $\mathcal{H} \leftarrow \emptyset$; scores $\mathcal{S} \leftarrow \emptyset$; cost $G \leftarrow 0$
for $t = 1$ **to** B **do**
  $a \leftarrow$ UCB descent of $\mathcal{T}$ (Eq. 1, values from Eq. 2; unvisited children ordered by Eq. 5)
  if $|\mathcal{H}| \geq w$ **and** $u \sim U(0,1) < p_m$ **then**
    $a \leftarrow$ mutate(best of 5 random members of $\mathcal{H}$)
  if $a \in \mathcal{T}$ **or** a invalid **then**
    $a \leftarrow$ random unevaluated architecture
  if $t > w$ **then**
    $\mathcal{S} \leftarrow \mathcal{S} \cup \{P(a)\}$ ▷ Eq. 3, no labels
    if $P(a) < q_{1-q}(\mathcal{S})$ **then**
      increment visit counts on the path of a; **continue**
  $A \leftarrow$ validation accuracy of a; $G \leftarrow G + \tau(a)$ ▷ real evaluation
  insert a in $\mathcal{T}$; back up max accuracy and visits; $\mathcal{H} \leftarrow \mathcal{H} \cup \{(a, A)\}$
  if t mod 50 = 0 **then**
    prune 15% lowest-value nodes (≥ 2 visits)
$a^* \leftarrow \arg\max_{(a,A)\in\mathcal{H}} A$
return $T(a^*)$, $|\mathcal{H}|$, G

3.5 *Optional LLM prior and pruning*

When enabled, an LLM receives the tokens fixed so far and the list of unvisited operations, and returns an integer score from 0 to 9 for each, under a strict JSON schema. Scores are converted to a distribution p_{LLM} and enter only the ordering of *never-visited* children,

$$Q_0(c) = \alpha_t \, p_{\text{LLM}}(o_c \mid \text{prefix}(c)), \qquad \alpha_t = \alpha / \sqrt{t'}, \quad (5)$$

where t' is the iteration rescaled to a budget of 100; once $\alpha_t < 0.02$ the prior is no longer queried. A zero-information prior reproduces the prior-off search exactly (12 of 12 seeds). In the headline system $\alpha = 0$; the prior is studied separately in section 5.9. Every 50 iterations the bottom 15% of internal nodes by backed-up accuracy, among those with at least two visits, are marked as pruned, never removing the last live child of a node. Pruning is a heuristic without guarantee.

3.6 *When should the gate penalise cost? A closed-form criterion*

Zero-cost proxies tend to favour larger networks, which take longer to train (section 5.6). An obvious correction is to penalise predicted cost D (here $\log(1 + \#\text{params})$) in the gate score, $S_\lambda = z(P) - \lambda z(D)$. To decide in advance whether this helps, we replace accuracy A, proxy P and predicted cost D by their normal scores and treat them as jointly Gaussian with pairwise correlations r_{PA}, r_{PD} and r_{DA} (a Gaussian copula). The correlation between the penalised score and accuracy is then

$$\rho(S_\lambda, A) = \frac{r_{PA} - \lambda \, r_{DA}}{\sqrt{1 + \lambda^2 - 2\lambda \, r_{PD}}}, \quad (6)$$

and its slope at $\lambda = 0$ is

$$\left.\frac{\partial \rho(S_\lambda, A)}{\partial \lambda}\right|_{\lambda=0} = r_{PA} \, r_{PD} - r_{DA}. \quad (7)$$

A small penalty improves the quality of what the gate admits only if $r_{DA} < r_{PA} r_{PD}$, i.e. only if accuracy is less coupled to cost than the proxy's own cost bias implies. If the slope is negative, the penalty can pay off only through the extra, cheaper evaluations it buys. To estimate that trade-off at a GPU-hour budget, we simulate gated random search in the fitted copula with the exact gate rule of Eq. (4). The derivation of Eq. (7) is given in Appendix A. All quantities are estimated from a sample of architectures before any search is run, so the prediction is made before the result is seen.

4 Experimental setup

4.1 *Benchmarks and data*

Table 2 lists the benchmarks: NAS-Bench-201 [8] (six edges with five operations each), NAS-Bench-101 [7] (cell graphs with up to seven nodes and nine edges), the size space of NATS-Bench [9] (five layer widths from $\{8, 16, \ldots, 64\}$) and HW-NAS-Bench [14], which adds the latency of every NAS-Bench-201 cell on six devices (edge GPU, Raspberry Pi 4, edge TPU, Pixel 3, Eyeriss and FPGA). Datasets are CIFAR-10/100 (C10/C100) [38] and ImageNet16-120 (IN16) [39]. All accuracies and training times are looked up, so nothing is trained during search; GPU-hours are the training times recorded in the benchmark of the architectures that a search actually evaluates. Tabular benchmarks are used deliberately: they make thousands of complete, seed-paired searches affordable, they remove training noise from the comparison of search strategies, and they let every claim be checked against the exhaustive ground truth of the space [7, 8, 18]. The three spaces were chosen to differ in the property that matters most for a proxy gate. NAS-Bench-201 is a small cell space and the main test bed. NAS-Bench-101 is a much larger cell space with variable graph topology, on which proxies are known to be weaker [13], so it tests transfer. The size space of NATS-Bench fixes the cell and varies only channel widths, so accuracy and cost are both driven by network width; it is the natural stress test for the claim that proxies detect cost (section 5.6).

How the data are kept. The raw benchmark files (about 6.5 GB) are converted once into small derived caches, the only files the experiments read (Table 1). Each record is keyed by a canonical code of the architecture (base 5 over the NAS-Bench-201 edges, base 8 over the NATS-Bench widths, the official module hash for NAS-Bench-101) and holds a validation accuracy for search, a test accuracy for reporting and a training time for cost: 200-epoch values averaged over training seeds for NAS-Bench-201, the 90-epoch schedule for NATS-Bench SSS and the mean of three 108-epoch repeats for NAS-Bench-101. The NAS-Bench-101 cache is read from the TFRecord file with a reimplementation of the official graph pruning and hashing, and its build refuses to finish unless all 423,624 module hashes are reproduced. HW-NAS-Bench latencies are aligned to NAS-Bench-201 indices, and the exact Pareto front of each device is precomputed. Raw proxy components are stored once per architecture (70,765 entries), so every search re-uses identi-

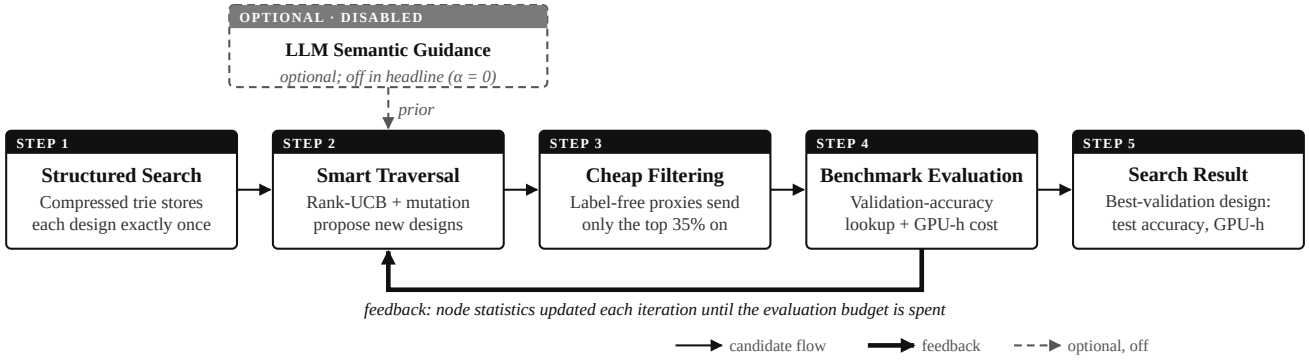


Figure 2. The ArchTree-RMG search loop in five steps. A compressed trie stores every evaluated design once (step 1); rank-normalised UCB descent and mutation propose a new design (step 2); the label-free proxy gate passes only the top 35% (step 3); passing designs are looked up in the benchmark and charged their training time (step 4); statistics are fed back up the trie until the budget is spent, and the best-validation design is reported (step 5). The LLM prior (dashed) is off in the headline system; Algorithm 1 gives the details.

Table 1. Derived data files read by the experiments.

File	Key	Fields per architecture	Size
NB201 cache (JSON)	base-5 index	valid, test, train time × 3 datasets	4.9 MB
NB101 cache (NPZ)	module hash	adjacency, ops, #params, valid/test/time × 3 repeats	22 MB
NATS-SSS cache (JSON)	base-8 code	valid, test, train time × 3 datasets; #params, FLOPs	12.8 MB
HW table (NPZ)	NB201 index	accuracy, latency on 6 devices	0.9 MB
True fronts (JSON)	device	Pareto-optimal (accuracy, latency) pairs	<3 kB
Proxy cache (JSON)	token string	raw E, S, T, size term	12 MB

Search uses the validation split; test accuracy is only reported. C10 validation in NB201 is the `cifar10-valid` split, as prescribed by the benchmark.

cal proxy values. All files are fixed by SHA-256 checksums, and every search writes one result row per run.

Figure 3 and Table 2 describe the spaces as data. Three properties shape the experiments. First, the spaces are top-heavy: the gap between the best architecture and the 99th percentile is only 0.6 points on NAS-Bench-201 CIFAR-10 and NATS-Bench SSS CIFAR-10, so differences between good search methods are naturally a few tenths of a point, and the statistics must be able to resolve them. Second, a long tail of poor architectures exists on NAS-Bench-201 (341 architectures below 20% on CIFAR-10, exactly the cells in which `none` operations disconnect the input from the output), which a cheap filter can remove. Third, accuracy and training time are positively correlated in every space (ρ = 0.27–0.65), so a filter that prefers accurate networks will also prefer expensive ones; this is why cost must be measured in GPU-hours rather than in evaluations.

4.2 *Baselines and arms*

Nine external baselines run on all three NAS-Bench-201 datasets with the same seeds: random search [17]; RE [15]; RE with a zero-cost warm-up (population seeded with the best of 1,000 proxy-scored architectures [10]); local search (LS) [19]; ENAS-style policy search [5]; MCTS-style search [21]; ZiCo-style proxy-guided sampling [28]; an RZ-style zero-cost mutation walk; and an LLM generator (LLM-NAS, `gpt-oss-20b` [40], eight architectures per call, best-so-far context) in the style of [29]. The re-implemented baselines carry the suffix "-style". They were chosen to cover the four families that dominate sample-based NAS (random, evolutionary, local, policy/tree search), the two existing ways of using zero-cost proxies (warm-up and proxy-guided sampling) and LLM generation. Internal arms include ArchTree without the gate, with a parameter-count gate, with a supervised gate and with cost-penalised gates (λ = 0.25 and 1), a flat per-position UCB with and without deduplication, and random and NSGA-II-style [41] search for the hardware study. Every proposer was also run with every gate (none, #params, supervised, label-free), so the effect of the gate can be separated from that of the proposer.

4.3 *Protocol and cost accounting*

NAS-Bench-201 budgets are 25, 50, 100 and 200 proposals, which spans the range used in the literature for this space (about 0.2–1.3% of it); NAS-Bench-101 budgets are 50, 100, 200 and 400 because the space is 27 times larger. Every configuration is run on 12 seeds, which, given the seed-to-seed spread in Table 5 (typically 0.1–0.8 points), resolves paired differences of a few tenths of a point; the base→RMG design changes were re-tested on 12 further held-out seeds that were never used for any design decision. In the *equal-GPU-hour* protocol each arm is run once with a large proposal budget

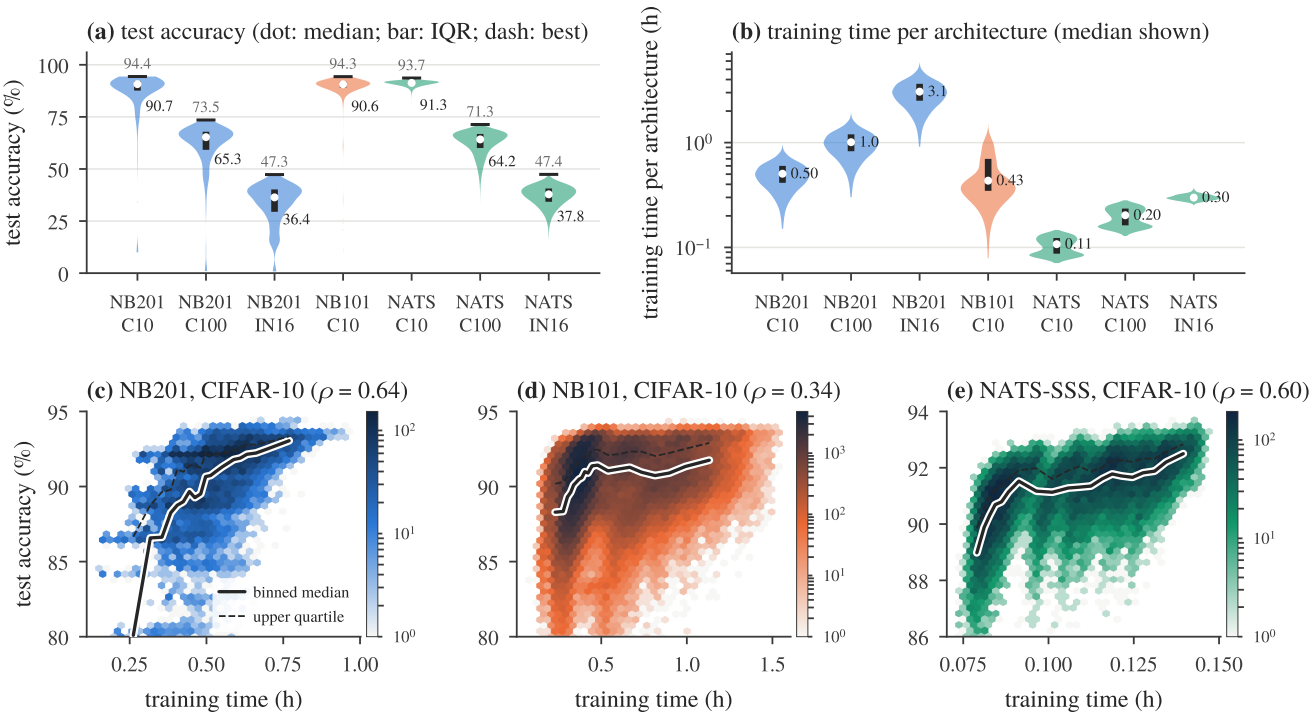


Figure 3. The three search spaces as data. (a) Test-accuracy distribution of every architecture (violin), with inter-quartile range (bar), median (dot) and best architecture (dash). (b) Distribution of the recorded training time of one architecture (log scale). (c)–(e) Joint density of training time and test accuracy on CIFAR-10 (hexagonal bins, log counts; the panels show the 89%, 98% and almost 100% of each space above the plotted accuracy floor). Accuracy rises with cost in all three spaces, but much less steeply in NAS-Bench-101, and NATS-SSS forms separate bands, one for each width of the most expensive layer.

Table 2. Statistics of the full search spaces (all architectures; accuracies in %, training time per architecture in GPU-hours).

Benchmark	Data	$\lvert\mathcal{A}\rvert$	Epochs	Test mean ± s.d.	Best	99th pct.	$\bar{\tau}$ [min, max] (h)	CV(τ)	ρ(valid, test) / ρ(test, τ)
NB201	C10	15,625	200	87.06 ± 12.94	94.37	93.78	0.51 [0.15, 0.96]	0.26	0.983 / 0.64
NB201	C100	15,625	200	61.39 ± 12.17	73.51	71.15	1.02 [0.30, 1.92]	0.26	0.993 / 0.65
NB201	IN16	15,625	200	33.57 ± 9.51	47.31	45.40	3.08 [0.91, 5.83]	0.26	0.994 / 0.62
NB101	C10	423,624	108	89.68 ± 5.80	94.32	93.51	0.54 [0.08, 1.53]	0.48	0.995 / 0.34
NATS-SSS	C10	32,768	90	91.09 ± 1.27	93.65	93.05	0.11 [0.07, 0.15]	0.18	0.963 / 0.60
NATS-SSS	C100	32,768	90	62.64 ± 5.63	71.34	69.76	0.20 [0.13, 0.28]	0.19	0.986 / 0.27
NATS-SSS	IN16	32,768	90	37.09 ± 4.75	47.40	44.97	0.30 [0.24, 0.38]	0.06	0.979 / 0.51

CV: coefficient of variation of training time. ρ: Spearman correlation over the whole space. The best test accuracy is not attainable by a validation-based search when validation and test rankings disagree.

and its evaluation history is cut at

$$H_k = k\,\bar{\tau}, \qquad \sum_{a \in \mathcal{E}} \tau(a) \le H_k, \qquad k \in \{25, 50, 100\}, \tag{8}$$

where $\bar{\tau}$ is the average time needed to train a single architecture of that benchmark and dataset (Table 2) and $\mathcal{E}$ is the set of evaluated architectures. Because no arm plans with the budget, truncation is equivalent to running with budget H_k. Fourteen arms on NAS-Bench-201 and NATS-Bench SSS and ten on NAS-Bench-101, with 12 seeds, give 1,128 runs. Proxy time is charged as well; at 0.28 s per scored proposal it adds at most 0.05% to a NAS-Bench-201 budget and of the order of 0.2% on NATS-Bench SSS, whose networks train fastest.

The work was organised in the stages of Table 3, each answering one question before the next was asked. In total 12,289 complete searches were run, all with fixed seeds and all logged; nothing reported in this paper is drawn from a run outside these files.

4.4 *Statistics*

Comparisons are paired by seed. We use two-sided Wilcoxon signed-rank tests [32] with Holm correction [33] within each comparison family and report a result as a win or loss only if $p_{\text{Holm}} < 0.05$. Across methods, we use the Friedman test

Table 3. Experimental campaign: stages, purpose and number of complete search runs.

Stage	Question answered	Runs
Baselines	reference performance (9 methods)	1,296
S1 ArchTree-base	does a deduplicating tree search work?	144
S2 + supervised gate	how much does a proxy gate save?	144
S4 ArchTree-RMG	headline system	144
Proposer × gate grid	is the gate portable? (6 proposers × 4 gates)	3,456
Held-out design ablation	which design change matters? (new seeds)	720
Trie vs flat bandit (A7)	hierarchy or deduplication?	432
Toggle ablations (A1–A6)	LLM, gate, inheritance, α, pruning, budget	792
UCB sensitivity	C and backup rule	1,008
Proxy subsets	which proxies in the gate?	612
LLM studies	speed, α sweep, model size	973
NAS-Bench-101	transfer to a larger space	576
Equal GPU-hours	fair-cost comparison, 3 benchmarks	1,128
Hardware (HW-NAS-Bench)	accuracy–latency search	864
Total		12,289

Counts are complete search runs recorded in the result files. S3 (ArchTree-RM) is part of the proposer × gate grid. The LLM knowledge probe adds 672 queries.

with blocks (dataset, budget, seed) and the Nemenyi critical difference (CD) [34]. Because a non-significant difference does not show that two methods are equal, two one-sided tests (TOST) [35] with margins of ±0.5 and ±0.25 points are reported as well, along with the probability that ArchTree beats a method, with stratified-bootstrap 95% confidence intervals [36]. Where an LLM-guided trajectory is identical for all seeds, paired differences that do not vary between seeds are not tested.

4.5 *Implementation*

The code is in Python 3.10 with PyTorch 2.5.1; proxies run on an NVIDIA RTX 2050 (4 GB). The LLM is `gpt-oss-20b` [40] served by Groq, with answers cached by prompt; no LLM-prior run used a fallback answer, whereas the LLM-NAS baseline replaced on average 0.37 unparsable architectures per run by random ones. Runs are deterministic: 1 and 3 workers give identical results (252/252 runs). Table 4 lists the settings, which were fixed before the final runs and are identical in all reported runs.

4.6 *Why these settings*

Each setting in Table 4 is either justified by a measurement on the full space or shown not to matter (Figure 4).

Pass rate. Ranking all 15,625 NAS-Bench-201 architectures by the label-free score and admitting the top fraction q gives the operating curve of Figure 4(a). The smallest q that keeps every architecture of the top 5% (by validation accuracy) is 34%, 34% and 29% on the three datasets, and at $q = 0.35$ the gate keeps 100% of both the top 1% and the top 5% on all three; on NATS-Bench SSS it keeps 98.5–100% of the top 1%. A tighter gate starts to lose good architectures (at $q = 0.2$ only 80% of the CIFAR-10 top 5% survive), while a looser gate only adds cost. The search applies the same quantile to the scores seen so far, which approximates this whole-space threshold once enough proposals have been scored.

Cost of what passes. Figure 4(b) shows the price of this choice. On NAS-Bench-201 the admitted 35% train 1.22 times longer than an average architecture, whereas the truly best 35% train 1.16–1.18 times longer; the proxy therefore over-pays by only 3–5%. On NATS-Bench SSS the admitted set is even cheaper than the true top set (1.02–1.05 against 1.03–1.12).

Proxy composition. The gate averages E, S and T and leaves out the size term C of the proxy cache. Among all 15 subsets of the four components used as gates (Figure 4(d)), the three-proxy set is among the best-ranked ($\tau = 0.66$, 0.71 and 0.62) and keeps 93–100% of the top 5%, whereas adding the size term lowers that recall on CIFAR-10 to 80%; no subset is significantly better than the four-component set (2 of 48 tests against it differ, both because SVD entropy alone is worse). Across the 15 subsets, a subset's rank correlation hardly predicts the accuracy of the search that uses it (Spearman 0.02, 0.15 and 0.54), whereas its recall of the top 5% does better on CIFAR-100 (0.77, $p < 0.001$): a gate must keep the best architectures, not rank the whole space. Leaving out an explicit size term is also what the criterion of section 3.6 recommends.

UCB, warm-up and pruning. Of 14 combinations of $C \in \{0.1, 0.2, 0.3, 0.5, 1, 1.414, 2\}$ with mean or max backup (Figure 4(c)), $C = 0.3$ with max backup has the highest mean accuracy, the others lying 0.02–0.31 points below, but none of the 78 differences is significant. Ten unconditional proposals give the running quantile and the tournament a minimum sample, and pruning thresholds from 0 to 35% change accuracy by less than the seed-to-seed spread.

5 Results

5.1 *Stage-wise development*

Figure 5(a)–(c) and the lower block of Table 5 follow the system through its four stages. S1, a deduplicating trie searched by UCB with raw reward and mean backup, already beats random search (94.11 against 93.92 on CIFAR-10 at budget 200) but stays below RE and LS. S2 adds a supervised zero-cost gate to S1: it cuts GPU-hours by 64–65%, but on CIFAR-100 its accuracy drops by 0.67 points, because a near-uniform tree descent fed with fewer evaluations finds fewer good regions. S3 replaces the raw reward by the rank reward and adds tournament mutation; it matches S1 on CIFAR-10, is 0.15 points lower on CIFAR-100 and 0.14 points higher on ImageNet16-120, but, without a gate, spends as much as RE. S4 adds the label-free gate to S3 and combines the two benefits: 94.28, 73.40 and 46.41 with 81–83 real evaluations and 47% of the GPU-hours of S3. In

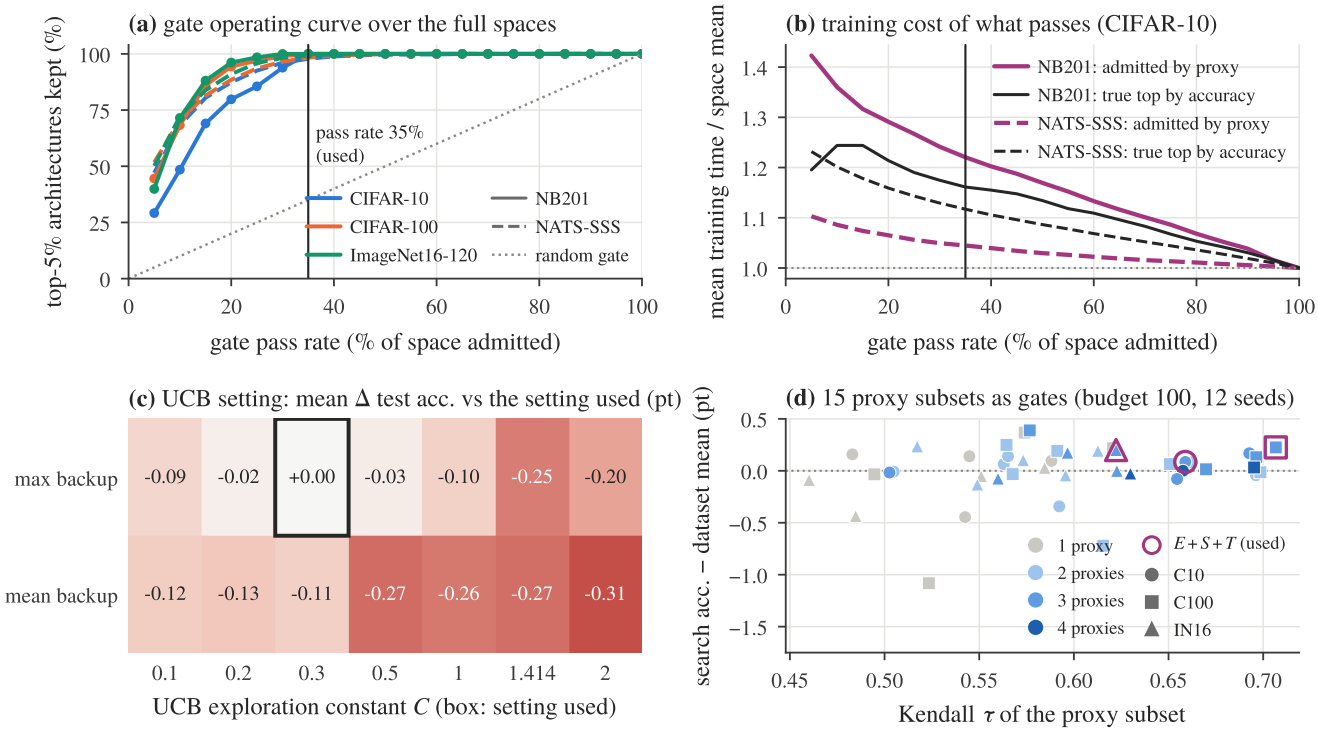


Figure 4. Evidence for the settings of ArchTree-RMG. (a) Share of the top-5% architectures (by validation accuracy) that a static label-free gate keeps when it admits a fraction q of the whole space; the vertical line marks $q = 0.35$ and the dotted diagonal is a random gate. (b) Mean training time of the admitted architectures relative to the space mean (red), compared with the truly best fraction q (black). (c) Mean change in test accuracy of each UCB setting relative to the one used, averaged over three datasets and two budgets (12 seeds each). (d) Each of 15 proxy subsets used as the gate: rank correlation against search accuracy at budget 100, relative to the mean over subsets; circles mark the subset used.

the accuracy–cost plane S4 traces the upper-left boundary in all three panels (S2 touches it once, on CIFAR-10 at about 24 GPU-hours); RE and LS reach final accuracies within −0.03 to +0.11 points of S4 only by spending 2.2–2.3 times as many GPU-hours.

The gate's savings grow with budget (Figure 5(d), (e)), because the running 35% threshold becomes more selective as better architectures accumulate. At a budget of 200 it removes 59–60% of real evaluations and 53% of GPU-hours on all three datasets, while test accuracy changes by +0.16, +0.43 and +0.02 points. GPU-hour savings are about six points below the evaluation savings since the architectures that pass the gate are larger and slower to train, as anticipated by Figure 4(b). Figure 5(f) shows the other source of waste: a flat bandit without memory spends 60–75% of its evaluations on repeats and an LLM generator 6% at budget 25 rising to 55% at budget 200, whereas the trie spends none.

5.2 *Proxy quality without labels*

Figure 6(a) shows the Kendall rank correlation τ [42] of each proxy with validation accuracy on 1,500 NAS-Bench-201 architectures. Single proxies reach $\tau = 0.49$–0.64. The label-free ensemble reaches 0.64, 0.68 and 0.63 on the three datasets (0.64, 0.67 and 0.64 over all 15,625 architectures), close to the supervised ensemble (0.67, 0.72, 0.63) that consumes 1,500 labels, and a search with the supervised gate is statistically indistinguishable from the label-free one (Table 5). The components are complementary (Figure 6(c)): SVD entropy correlates only 0.47 with SynFlow, and averaging lifts the correlation with accuracy to 0.83, above any single proxy (0.73–0.78). On samples of 2,000 random architectures, proxy quality is lower on NAS-Bench-101 ($\tau = 0.33$ label-free, 0.50 supervised) and intermediate on NATS-Bench SSS (0.53–0.63).

5.3 *Convergence*

Figure 7 plots simple regret against real evaluations. After the first ten or so evaluations the curves separate: random search follows its analytic expectation, LLM-NAS starts lowest (its first proposals are good) but flattens as it begins to repeat itself, and the S4 curve falls fastest of all ArchTree stages. At its last real evaluation (at most 88, 98 and 102 on the three datasets) S4 is within 0.04, 0.08 and 0.15 validation points of the best architecture of the space, against 0.41, 1.55 and 1.22 for random search after 200 evaluations and 0.02, 0.17 and 0.15 for RE. In Figure 7(d), ArchTree-RMG comes within 0.5 points of the optimum after 25, 48 and 31 real evaluations, against 44, 88 and 85 for RE and 31,

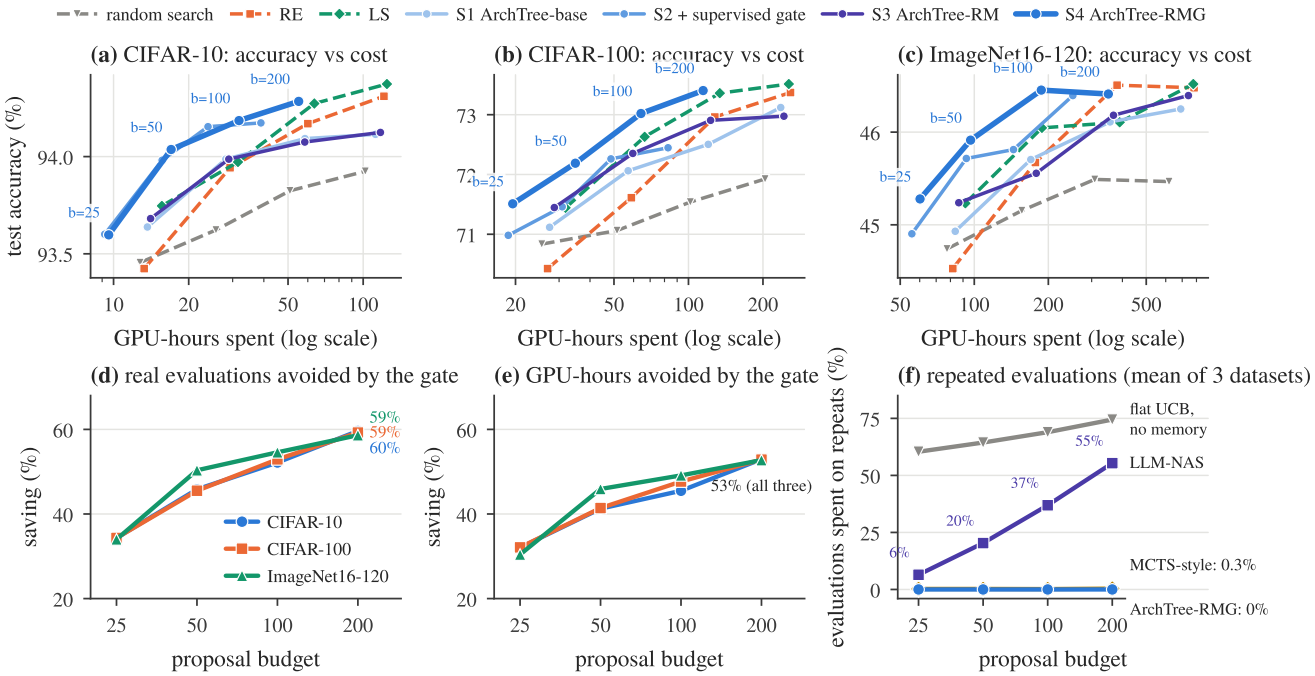


Figure 5. Stage-wise development on NAS-Bench-201 (12 seeds). (a)–(c) Test accuracy against GPU-hours spent, one marker per proposal budget (25, 50, 100, 200; labelled for S4); further up and to the left is better. (d), (e) Real evaluations and GPU-hours avoided by the label-free gate when it is added to ArchTree-RMG at the same proposal budget. (f) Share of real evaluations spent on architectures that had already been trained. Stages: S1 = ArchTree-base (trie + raw-reward UCB); S2 = S1 + supervised gate; S3 = ArchTree-RM (rank UCB + mutation, no gate); S4 = ArchTree-RMG (S3 + label-free gate).

69 and 81 for LS, while random search, MCTS-style search and LLM-NAS reach this level only on CIFAR-10, if at all. Only the two methods that pre-screen 1,000 proxy-scored candidates before their first evaluation (ZiCo-style and RE with ZC warm-up) get there sooner, which again points to the value of proxies as a filter.

5.4 *Comparison with baselines at a fixed proposal budget*

Table 5 reports test accuracy at a budget of 200, together with real evaluations and GPU-hours. Baselines evaluate all 200 proposals; ArchTree evaluates about 81–83 of them and uses 44–46% of the GPU-hours of ungated RE. Figure 8 puts accuracy, cost, sample efficiency and the statistics on one page. Local search and RE with ZC warm-up lead on accuracy and rank, ArchTree-RMG leads on cost with 41% of the evaluations and 38–56% of the GPU-hours of the baselines, and only the proxy-screening methods reach low regret with fewer real evaluations. The 108 paired tests (nine baselines, three datasets, four budgets) break down as follows. With Holm correction within each baseline, ArchTree records 20 wins, 88 ties and no losses; the wins are against random search (7), MCTS-style (6), ENAS-style (3), LLM-NAS (3) and ZiCo-style (1). Without correction there are 32 wins and 6 losses, three of them against RE with zero-cost warm-up at small budgets; at budget 25 this method reaches 93.91, 72.00 and 46.11 against 93.60, 71.51 and 45.28 for ArchTree-RMG, so the gate's warm-up of ten unfiltered proposals is a weakness at very small budgets. Equivalence within ±0.5 points is shown by TOST in 27 of the 108 cells. With Holm correction over all 108 tests jointly, every comparison is a tie, both at the nominal budget and when every baseline receives at least ArchTree's own number of real evaluations; without correction, that strict equal-evaluation comparison gives 33 wins, 70 ties and 5 losses. In the Friedman ranking at full budget (10 methods, 144 blocks, CD = 1.13), ArchTree ranks 4.41, behind RE with zero-cost warm-up (3.67) and local search (3.90) but within the CD of both, while using about 40% of their evaluations.

5.5 *Equal GPU-hours*

Comparisons at an equal number of proposals favour any method that evaluates fewer, but possibly more expensive, architectures. Figure 9 therefore gives every arm the same training budget H_k (Eq. 8). Across 13 comparison arms and up to 21 cells (seven benchmark–dataset pairs, three budgets), ArchTree with the label-free gate records 57 wins, 204 ties and *no loss* (Holm within arm). The largest mean margins are over the cost-penalised gates (+1.36 points for random search and +1.01 for ArchTree-RM with $\lambda = 1$; 15 and 16 wins) and over random search without the gate (+0.67, 10

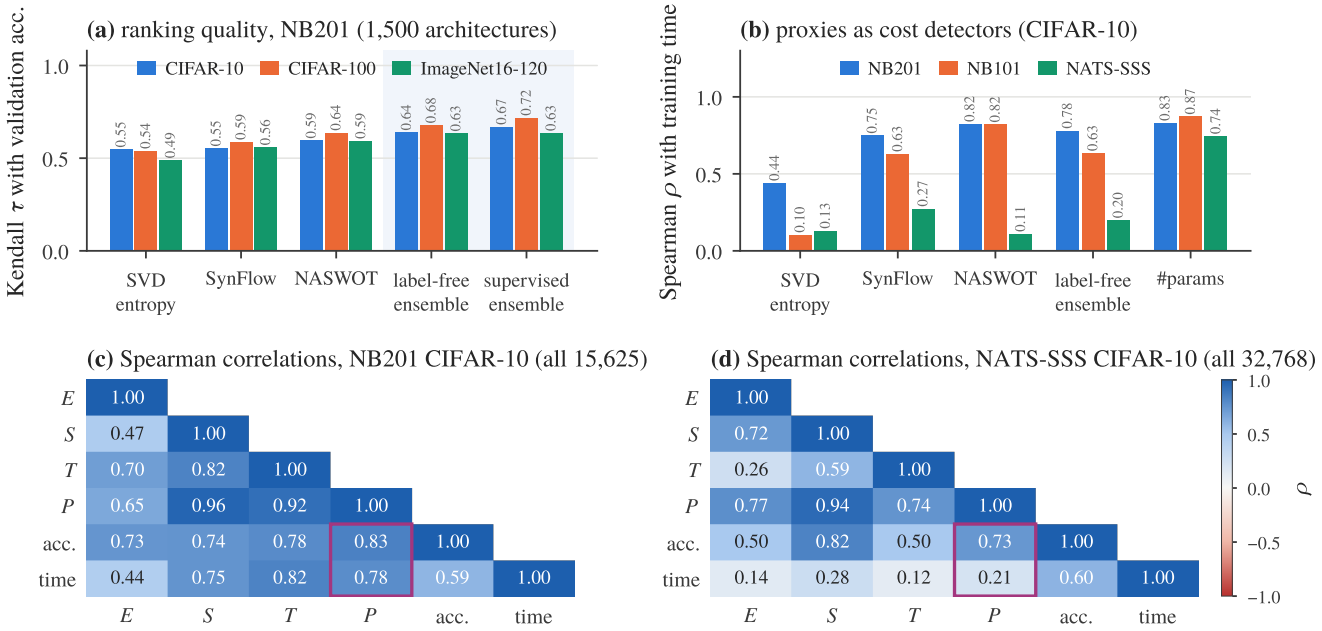


Figure 6. Zero-cost proxies. (a) Kendall τ between each proxy and validation accuracy (1,500 NAS-Bench-201 architectures); the label-free ensemble uses no labels and almost matches the supervised one. (b) Spearman ρ between proxy score and recorded training time in the cell spaces NB201 (all 15,625 architectures) and NB101 (2,000), and the width space NATS-SSS (2,000). (c), (d) Spearman correlations between the three proxies, the label-free score P, validation accuracy and training time over the complete spaces; the boxed cells show that P is as strongly related to training time as to accuracy on NB201, but only to accuracy on NATS-SSS.

Table 4. Settings of ArchTree-RMG (identical in all reported runs) and the evidence for each.

Choice	Evidence (section)
Trie	no repeated evaluation by construction; flat bandit wastes 73–74% (5.8)
Rank reward	raw reward gives near-uniform descent; the base design is 0.63 pt worse on held-out seeds (5.8)
Mutation $p_m = 0.5$	carries most of the accuracy; mutation-only is level with ArchTree-RMG (5.8)
$C = 0.3$, max backup	best of 14 settings; 0/78 significant differences (Fig. 4c)
Gate $q = 0.35$	smallest round rate keeping all top-5% NB201 architectures (Fig. 4a)
$E + S + T$, no labels	τ within 0.04 of the supervised ensemble; size term lowers recall (Fig. 4d)
No cost penalty	criterion slope < 0 on 7/7 pairs; penalty loses 16/21 cells (5.6)
LLM prior off	faster start only; no final-accuracy gain (5.9)
Pruning 15%	0–35% indistinguishable (5.8)
Warm-up 10, tournament 5	minimum sample for the quantile and the tournament
z-stats from 1,500 archs	unlabelled sample; weights $E = S = T = \frac{1}{3}$
Seeds	12 main + 12 held-out for design checks

wins); against ungated RE, LS and ArchTree-RM the margins are +0.36, +0.16 and +0.38 points with at most two wins each. The wins are concentrated against the cost-penalised gates and against random search with or without the gate; against RE and LS with the same gate, and against RE with zero-cost warm-up, all 21 cells are ties.

Friedman ranks (Figure 10) give the same picture. On NAS-Bench-201 ArchTree has the lowest average rank (4.75), essentially tied with RE plus zero-cost warm-up (4.84; CD = 1.91). On NAS-Bench-101 it is second (4.38, behind LS with the gate at 3.11; CD = 2.26) and on NATS-Bench SSS fourth (5.99, behind LS with the gate at 4.19), in both cases within the CD; section 5.11 gives the per-run probabilities and equivalence tests.

Under the same label-free gate at a fixed proposal budget, ArchTree matches RE in all 12 NAS-Bench-201 cells while using 19–23% fewer GPU-hours (55, 115 and 351 GPU-h against 72, 146 and 434 at budget 200) and has no loss in 240 same-gate comparisons against five proposers under four gates; the portability of the gate itself is analysed in section 5.7.

5.6 *Why the gate works: proxies as cost detectors*

Figure 6(b) shows that, in the cell spaces, the label-free ensemble correlates with training time almost as strongly as parameter count does (Spearman $\rho = 0.78$ on NAS-Bench-201 and 0.63 on NAS-Bench-101; NASWOT alone 0.82). Accordingly, the top 35% of NAS-Bench-201 architectures by proxy cost 1.22 times the mean to train. However, the truly best 35% by validation accuracy cost 1.16–1.18 times

Table 5. NAS-Bench-201, budget 200 proposals: test accuracy (%) of the best-validation architecture, mean ± standard deviation over 12 seeds, with real evaluations and GPU-hours (C10 / C100 / IN16). Best accuracy per column in bold.

Method	Real evals	GPU-h	CIFAR-10	CIFAR-100	ImageNet16-120
Random search [17]	200	102 / 204 / 619	93.92 ± 0.21	71.92 ± 0.81	45.47 ± 0.80
MCTS-style [21]	200	103 / 207 / 630	93.89 ± 0.20	71.94 ± 0.87	45.44 ± 0.65
ENAS-style [5]	200	115 / 236 / 717	94.09 ± 0.26	72.59 ± 0.81	46.16 ± 0.36
LLM-NAS (`gpt-oss-20b`)	200	140 / 274 / 825	94.28 ± 0.08	72.14 ± 0.43	45.37 ± 0.36
ZiCo-style [28]	200 + proxy	138 / 275 / 835	94.29 ± 0.12	72.97 ± 0.43	46.28 ± 0.57
RZ-style ZC walk	200 + proxy	149 / 298 / 906	94.33 ± 0.07	73.39 ± 0.19	46.32 ± 0.18
RE [15]	200	121 / 258 / 782	94.31 ± 0.13	73.37 ± 0.21	46.48 ± 0.17
RE + ZC warm-up [10]	200 + proxy	125 / 263 / 820	94.28 ± 0.19	73.36 ± 0.23	46.34 ± 0.36
Local search [19]	200	124 / 254 / 777	**94.37 ± 0.00**	**73.51 ± 0.00**	**46.52 ± 0.29**
S1: ArchTree-base (trie + UCB)	200	112 / 236 / 690	94.11	73.12	46.25
S2: S1 + supervised gate	62–65	39 / 83 / 252	94.17	72.45	46.39
S3: ArchTree-RM (no gate)	200	117 / 243 / 742	94.12	72.97	46.39
ArchTree-RM + supervised gate (1,500 labels)	76–81	54 / 112 / 348	94.26	73.27	46.43
S4: ArchTree-RMG (ours)	81–83	**55 / 115 / 351**	94.28 ± 0.13	73.40 ± 0.20	46.41 ± 0.18
Best architecture in the space (by validation)	–	–	94.37	73.51	46.20

"+ proxy": every proposal is also proxy-scored. GPU-h are recorded 200-epoch training times of the evaluated architectures; the S2 GPU-h is low because its supervised gate admits fewer proposals. On ImageNet16-120 several methods exceed the best-validation architecture in test accuracy because validation and test rankings differ slightly.

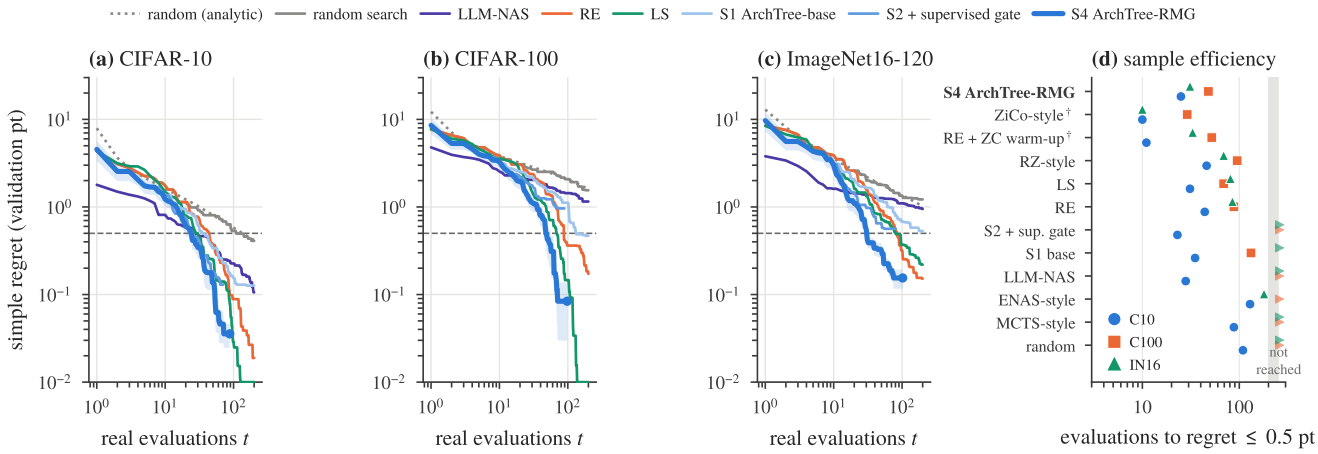


Figure 7. Convergence on NAS-Bench-201 at a budget of 200 proposals: simple regret (best validation accuracy in the space minus the best found so far, mean of 12 seeds; band: ±1 standard error for S4) against the number of real evaluations. Gated stages stop earlier on the horizontal axis because they evaluate fewer proposals; the dotted line is the exact expectation for uniform random search without replacement and the dashed line marks 0.5 points. (d) Real evaluations after which the mean regret first falls to 0.5 points (▷: not reached within 200); †these methods pick their first evaluations from 1,000 proxy-scored candidates. S1 = ArchTree-base, S2 = S1 + supervised gate, S4 = ArchTree-RMG.

the mean, because accuracy itself rises with network size, so only 3–5% of the extra cost is wasted (Figure 4(b)). In the width space of NATS-Bench SSS the ensemble is only weakly cost-related (ρ = 0.20–0.21), whereas parameter count still is (ρ = 0.68–0.74). The components differ as well: SVD entropy is nearly cost-neutral (its top 35% train 1.13 times the mean, below the 1.16 of the truly best 35%), whereas NASWOT over-pays by 6%. On NAS-Bench-101 the over-payment is much larger: the admitted 35% train 1.35 times the mean against 1.14 for the truly best, which explains why the GPU-hour advantage over RE shrinks there; Figure 6(d) shows that there the proxy follows accuracy (ρ = 0.73) but not cost.

Table 6 applies the criterion of Eq. (7) to all seven benchmark–dataset pairs, using parameter count as the predicted cost D. The slope is negative everywhere, so a cost penalty is predicted to lower the quality of admitted architectures. The two regimes differ: in the cell spaces the slope is slightly negative (−0.02 to −0.05), because the proxy already tracks cost, whereas on NATS-Bench SSS it is strongly

	ACCURACY			COST		EFFICIENCY		STATISTICS vs RMG			
	C10 ↑	C100 ↑	IN16 ↑	Real evals ↓	GPU-h (3 sets) ↓	Evals to regret ≤ 0.5 ↓	Repeated evals % ↓	Friedman rank ↓	P(RMG better)	Holm W/T/L	
Random search	93.92	71.92	45.47	200	925	109	**0.0**	7.85	0.81	7/5/0	*baselines*
MCTS-style	93.89	71.94	45.44	200	940	88	0.3	7.97	0.85	6/6/0	
ENAS-style	94.09	72.59	46.16	200	1068	129	**0.0**	6.57	0.77	3/9/0	
LLM-NAS	94.28	72.14	45.37	200	1238	28	55.4	6.70	0.75	3/9/0	
ZiCo-style	94.29	72.97	46.28	200	1248	**10**	**0.0**	4.47	0.51	1/11/0	
RZ-style	94.33	73.39	46.32	200	1353	46	**0.0**	4.41	0.50	0/12/0	
RE	94.31	73.37	46.48	200	1162	44	**0.0**	5.07	0.57	0/12/0	
RE + ZC warm-up	94.28	73.36	46.34	200	1208	11	**0.0**	**3.67**	**0.41**	0/12/0	
Local search	**94.37**	**73.51**	**46.52**	200	1155	31	**0.0**	3.90	0.43	0/12/0	
S1 ArchTree-base	94.11	73.12	46.25	200	1038	35	**0.0**	–	–	–	*ArchTree stages*
S3 ArchTree-RM	94.12	72.97	46.39	200	1102	–	–	–	–	–	
S4 ArchTree-RMG	94.28	73.40	46.41	**82**	**520**	25	**0.0**	4.41	ref.	ref.	

Figure 8. Method scorecard on NAS-Bench-201 at a budget of 200 proposals (12 seeds). Each cell shows the raw value; shading ranks the methods within each column (darker = better, best in bold; grey = not available). Accuracy is the test accuracy of the best-validation architecture; GPU-h is summed over the three datasets; evaluations to regret ≤0.5 and repeated evaluations are for CIFAR-10 and the mean of the three datasets, respectively. Statistics compare each baseline with ArchTree-RMG over 12 dataset–budget cells: Friedman average rank (10 methods, 144 blocks, CD = 1.13), bootstrap probability that ArchTree-RMG wins a paired run (darker = stronger rival) and Holm-corrected wins/ties/losses of ArchTree-RMG. S1–S4: development stages (section 3).

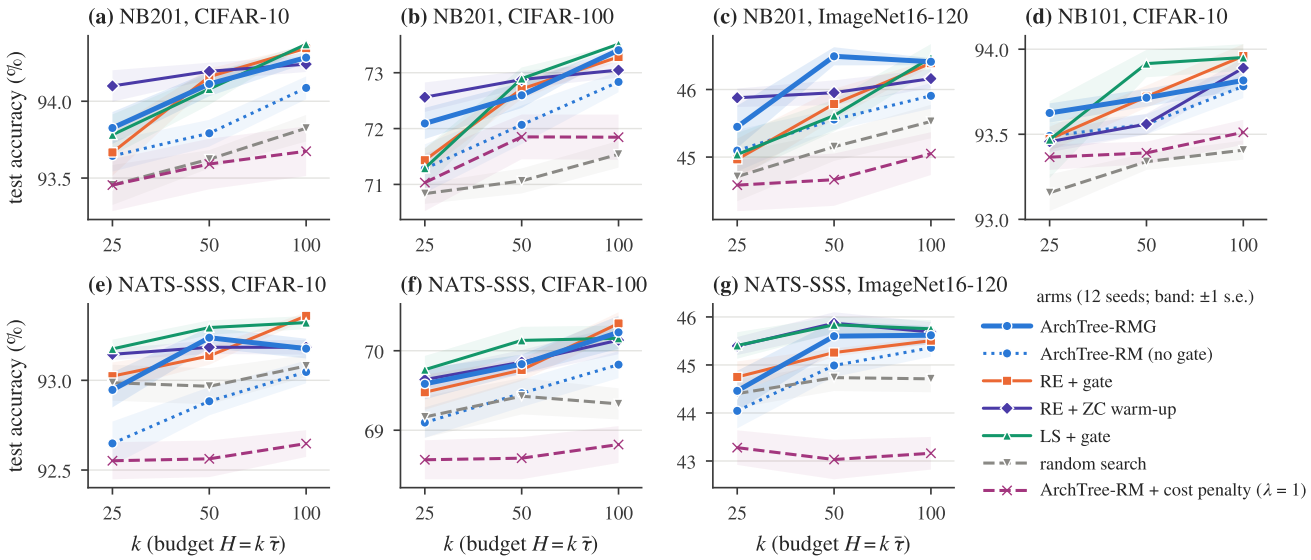


Figure 9. Test accuracy of the best-validation architecture found within an equal GPU-hour budget $H = k\bar{\tau}$ (mean ± standard error, 12 seeds). Solid lines use the label-free gate (or a ZC warm-up); dotted and dashed lines are ungated or cost-penalised. The gate lifts every proposer, while penalising cost lowers accuracy on all seven benchmark–dataset pairs.

negative (−0.19 to −0.39), because accuracy is tightly coupled to width (r_{DA} = 0.75–0.89). Figure 11(b) shows what this means for the gate: as λ grows, the correlation of the admitted score with accuracy falls monotonically on every pair, from 0.82–0.86 to 0.22–0.24 at $\lambda = 1$ on NAS-Bench-201 and to between −0.18 and +0.12 on NATS-Bench SSS, where a gate with $\lambda = 1$ is barely better than random. The measured outcome agrees (Figure 11(c)). With $\lambda = 1$, ArchTree loses significantly to its own unpenalised gate in 16 of the 21 cells without a single win, and its change in test accuracy is negative in all 21 cells (random search with $\lambda = 1$: 18 of 18). Over all penalised arms, predicted and measured changes share the same sign in 72% (ArchTree) and 69% (random) of cells, with Spearman correlations of +0.78 and +0.86. The copula simulation reproduces the level of the unpenalised gate closely (predicted 93.89, 71.96 and 45.67 against measured 93.92, 71.92 and 45.45 for gated random search on NAS-Bench-201 at $k = 100$) but under-

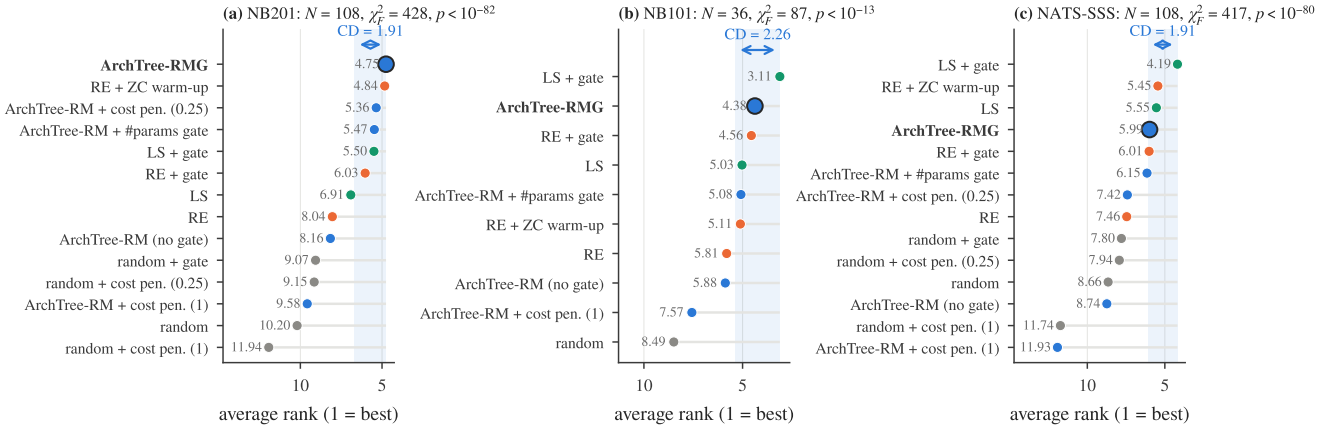


Figure 10. Average Friedman rank at equal GPU-hours (blocks = dataset × budget × seed; numbers give the rank). The shaded band spans the Nemenyi critical difference (CD) from the best arm; arms inside it are not significantly worse than the best. Colours mark the proposer family; ArchTree-RMG (large marker) is inside the band on all three benchmarks.

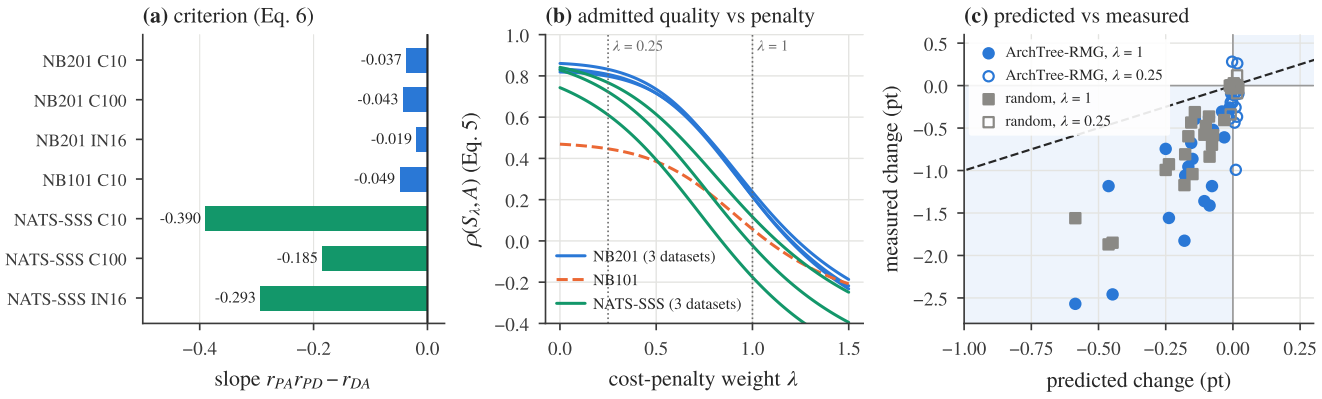


Figure 11. (a) Criterion slope $r_{PA}r_{PD} - r_{DA}$ for each benchmark–dataset pair; all are negative, predicting that a cost penalty hurts (blue: cell spaces; green: width space). (b) Correlation between the penalised gate score and accuracy as a function of λ (Eq. 6) with the fitted correlations; dotted lines mark the two penalties tested. (c) Predicted against measured change in test accuracy caused by a cost-penalised gate at equal GPU-hours (one point per benchmark, dataset and budget). Points in the shaded quadrants have the predicted sign; the ordering is well predicted, but measured losses are several times larger than predicted (dashed line: $y = x$).

predicts the size of the losses from a penalty by a factor of several, so we use the criterion for its sign and ranking, not for magnitudes.

5.7 *Transfer to NAS-Bench-101 and portability of the gate*

Figure 12(a) repeats the same-budget experiment on NAS-Bench-101 (576 runs) with three proposers and four gates. Proxies are weaker here, yet the label-free gate still reduces real evaluations by 59% and GPU-hours by 47% at budget 400 with no loss of accuracy relative to ungated ArchTree. Across 16 gate–budget cells, ArchTree beats random search in 14 and ties in 2. Against RE it ties in 15 and loses one (parameter-count gate, budget 100, −0.31 points). At budget 400 the label-free gate gives 93.96 for ArchTree, 94.07 for RE and 93.68 for random search with 163 real evaluations; RE is numerically 0.1–0.2 points ahead, and ArchTree's GPU-hour advantage over RE shrinks to about 5% (105 against 110 GPU-h). Over the cost range they share, the gated curves lie above their ungated counterparts, so the gate transfers even where proxies rank poorly. This is consistent with the data (Table 2): accuracy is less coupled to cost in NAS-Bench-101 ($\rho = 0.34$), so a cost-correlated proxy has less to offer.

Figure 12(b) summarises the portability of the gate at equal GPU-hours. Adding it raises test accuracy in 25 of 27 proposer–benchmark cells, by 0.38 points on average for ArchTree, 0.26 for RE, 0.22 for random search and 0.19 for LS; the two exceptions (−0.07 and −0.02, RE and LS on NAS-Bench-201 ImageNet16-120) are within noise. Figure 12(c) shows how the gain is obtained. In the cell spaces

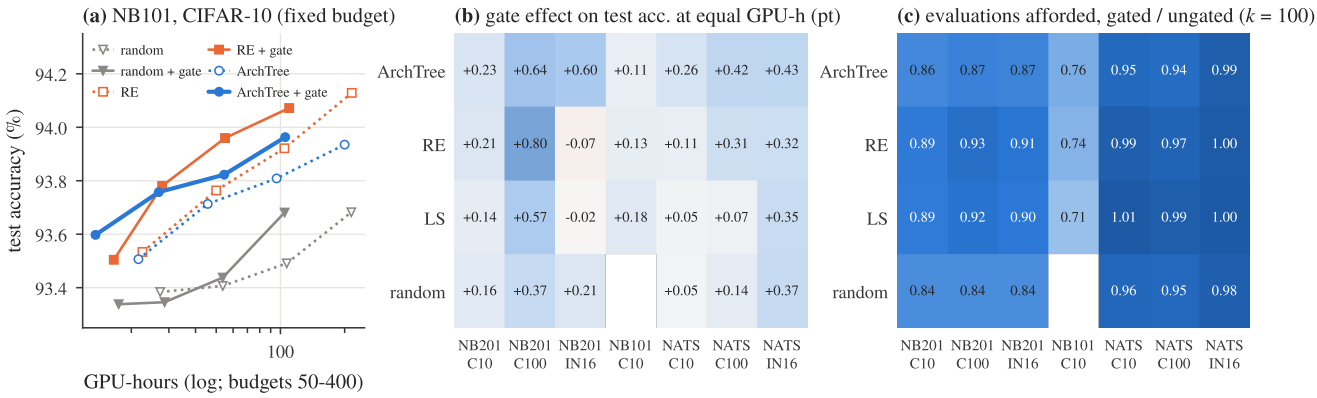


Figure 12. Transfer and portability of the gate. (a) NAS-Bench-101: test accuracy against GPU-hours at budgets 50–400 for three proposers with (solid, filled) and without (dotted, open) the label-free gate (12 seeds). (b) Change in test accuracy at equal GPU-hours when the label-free gate is added to each proposer, averaged over $k \in \{25, 50, 100\}$ (blank: arm not run). (c) Ratio of real evaluations made within H_{100} with and without the gate; values below 1 mean that the gate buys fewer, more expensive architectures.

Table 6. Gate-cost criterion (Eq. 7) with normal-score correlations, and the measured effect of a $\lambda = 1$ cost penalty on ArchTree at equal GPU-hours.

Bench.	Data	r_{PA}	r_{PD}	r_{DA}	Slope	Measured
NB201	C10	0.83	0.79	0.69	−0.04	3 L / 0 T
NB201	C100	0.86	0.79	0.72	−0.04	1 L / 2 T
NB201	IN16	0.82	0.79	0.66	−0.02	2 L / 1 T
NB101	C10	0.47	0.82	0.44	−0.05	1 L / 2 T
NATS	C10	0.74	0.67	0.89	−0.39	3 L / 0 T
NATS	C100	0.84	0.67	0.75	−0.19	3 L / 0 T
NATS	IN16	0.83	0.67	0.85	−0.29	3 L / 0 T

L/T: losses/ties of the penalised gate against the plain gate over $k \in \{25, 50, 100\}$ (Holm); no cell shows a win. NATS: NATS-Bench SSS.

the gate spends the same budget on 71–93% as many evaluations, because the architectures it admits are larger; on NATS-Bench SSS it spends it on 94–101% as many, because there the proxy is not cost-biased. In both regimes accuracy improves, which is the behaviour the cost-detector analysis of section 5.6 predicts.

5.8 *Ablations: what the parts contribute*

Tree structure versus deduplication. Replacing the trie by a flat bandit over the six positions (Figure 13(a)) shows where the benefit comes from. Without memory, the flat bandit spends 73–74% of its real evaluations at budget 200 on architectures it has already trained, and the trie beats it in 3 of 12 cells before and 1 after Holm correction (+0.53 points on average). With set-based deduplication added to the flat bandit, the difference falls to +0.06 points and 0 of 12 cells. The gain therefore comes from deduplication, not from the hierarchy. The landscape explains why (Figure 13(c)): the first one to three tokens of a NAS-Bench-201 architecture explain only 6–24% of the variance in accuracy (η^2), rising to 47–59% at depth four, so statistics stored near the root carry little information.

Design changes on held-out seeds. On 12 seeds not used for any design decision (Figure 13(b)), ArchTree-base is 0.63 points worse than ArchTree-RMG on average (significantly so in 3 of 12 cells), the rank reward alone is 0.18 points worse, mutation alone is level (+0.03), and RE with the same gate is level (+0.13, all ties), while ArchTree-RMG needs 12–19% fewer GPU-hours than RE at budget 200. The accuracy of ArchTree-RMG thus comes mainly from its evolutionary step, while the trie and the gate reduce what that accuracy costs.

Robustness. The UCB constant, the backup rule, the proxy weights and the pruning threshold have no significant effect (section 4.6), and the simulated weight inheritance has, by construction, no effect on a tabular benchmark.

5.9 *What does the LLM prior add?*

With the prior enabled ($\alpha = 0.3$), the search comes within one point of its final validation accuracy after 5.0 instead of 15.5 evaluations on CIFAR-10 and 15.6 instead of 30.8 on CIFAR-100 ($p_{\text{Holm}} = 0.047$ in both cases, Cohen's $d \approx 1.0$; Figure 14(a), (b)). At ten minutes per evaluation this is 0.8 instead of 2.6 hours on CIFAR-10; the model's own response time (on average at most 23 s per run when answers are not cached) is negligible next to a single evaluation. On ImageNet16-120 the prior is not faster (23.2 against 17.8 evaluations, not significant). The effect is confined to the start of the search: on final accuracy every comparison is a tie, and against no prior $|d| \leq 0.35$ (Figure 14(b)). The sweep of α (Figure 14(c); budgets 50 and 100, 12 seeds) points the same way. At $\alpha \geq 0.2$ the prior saturates and the runs become identical; at that level it changes test accuracy by +0.85 and +0.29 points on CIFAR-100, +0.20 and +0.10 on CIFAR-

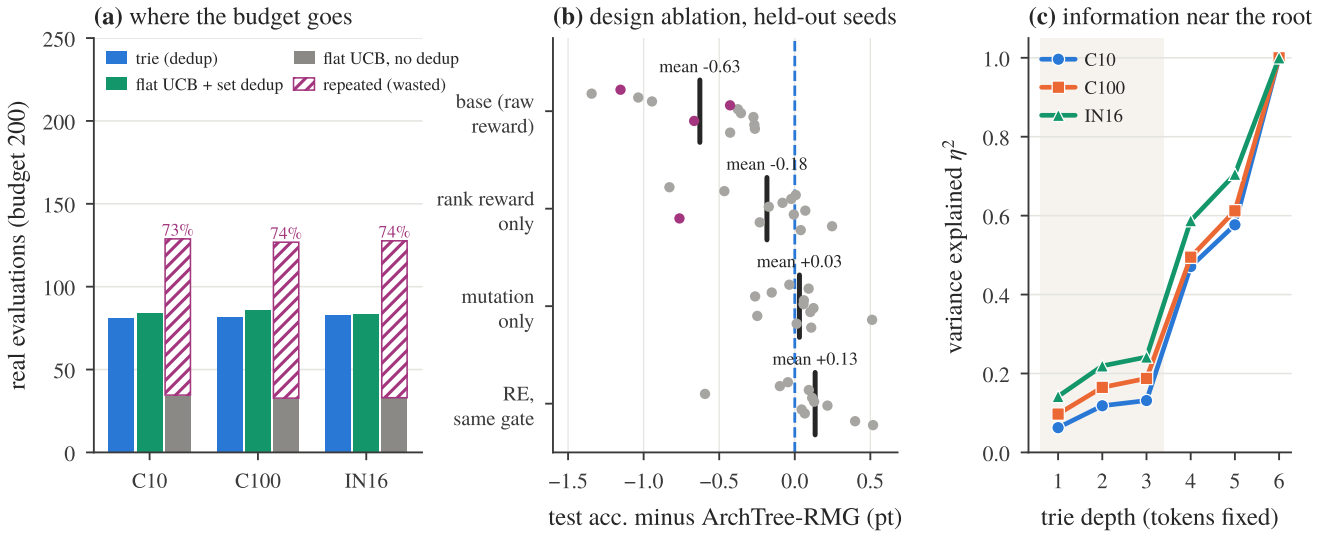


Figure 13. (a) Real evaluations at a budget of 200 proposals for the trie and for a flat per-position UCB with and without deduplication; hatched segments are evaluations of already-trained architectures. (b) Held-out-seed design ablation: test accuracy of each variant minus ArchTree-RMG in each of the 12 dataset–budget cells (dots; red marks cells where ArchTree-RMG is significantly better after Holm correction) and the mean over cells (bar). (c) Share of the variance of NAS-Bench-201 validation accuracy explained by the first d tokens; the shaded depths are where a tree would have to share statistics.

10 and −0.04 and −0.01 on ImageNet16-120 (budget 50 and 100). Of 24 distinct settings, 17 favour the prior, but none of the 48 tests is significant after Holm correction. A larger model (`gpt-oss-120b`) ranks operations slightly better (ρ = 0.61 against 0.60) but does not search better.

Three controls address what the model knows (Figure 14(d)). When operation names are replaced by arbitrary labels, the model's ranking of operations is only slightly better than chance (ρ = 0.14–0.19; significant but small, d = 0.3–0.4). When the operations are described in plain words without NAS-Bench-201 names, the ranking is *better* than with the benchmark's own names (ρ = 0.64–0.70 against 0.50–0.57), which argues against recall of memorised benchmark tables. A knowledge-free prior that scores operations by their size ranks single operations best of all (ρ = 0.73–0.76), yet it searches worse: it is no faster than no prior on CIFAR-10 and is significantly slower on ImageNet16-120 (+13.5 evaluations, p_{Holm} = 0.018), whereas the LLM prior is faster than the size prior on CIFAR-10 (p_{Holm} = 0.003, d = 4.0) and has a larger area under the early curve on CIFAR-10 and ImageNet16-120 (d = 0.93 and 0.92, both significant). The LLM thus adds more than a size rule, but only early.

5.10 *Hardware-aware search*

The equal-GPU-hour study measures the cost of *finding* a network; on an edge device, the cost that matters is the latency of *running* it. HW-NAS-Bench tests whether the loop extends to a second objective on the same 15,625 cells and whether a front found for one device is useful on another. A latency-penalised trie search is compared with random and NSGA-II-style search. It uses rank-normalised UCB with max backup on the reward $A - 5\min(\ell/50\,\text{ms}, 1)$, where ℓ is the latency, and has no gate and no mutation, so that the objective alone is changed; it is denoted ArchTree-HW. The comparison runs at budgets 25–200 (12 seeds, 864 runs) using hypervolume (HV) [43] and inverted generational distance (IGD) against the exact fronts.

Table 7. Accuracy–latency search on HW-NAS-Bench at budget 200 (CIFAR-10, mean of 12 seeds).

Device	HV ($\times 10^3$) AT / Rnd / NSGA	IGD AT / Rnd / NSGA	$\lvert F\rvert$ AT	$\lvert F^*\rvert$	HV / HV* AT
Edge GPU	91.35 / 91.06 / 91.18	0.22 / 0.38 / 0.17	12.6	24	0.995
RPi 4	91.00 / 90.88 / 91.16	2.04 / 1.31 / 1.78	18.4	48	0.991
Edge TPU	91.38 / 91.16 / 91.40	0.22 / 0.26 / 0.15	13.3	43	0.991
Pixel 3	91.24 / 91.05 / 91.25	0.69 / 0.80 / 1.44	18.0	55	0.990
Eyeriss	91.33 / 91.03 / 91.26	0.51 / 0.64 / 0.34	11.0	17	0.971
FPGA	91.40 / 91.07 / 91.30	0.55 / 0.64 / 0.35	11.1	11	0.961

AT: ArchTree-HW (latency-penalised trie search, no gate); Rnd: random; NSGA: NSGA-II-style. $\lvert F\rvert$: size of the found front; $\lvert F^*\rvert$: size of the true front; HV/HV*: share of the attainable hypervolume. All 96 HV and IGD comparisons (6 devices, 4 budgets, 2 rivals, 2 metrics) are ties after Holm correction.

The found fronts trace the true fronts closely (Figure 15) and reach 96–99.5% of the attainable hypervolume at budget 200, up from 85–94% at budget 25. We call this share HV/HV* the *hardware-aware coverage* of a search: the frac-

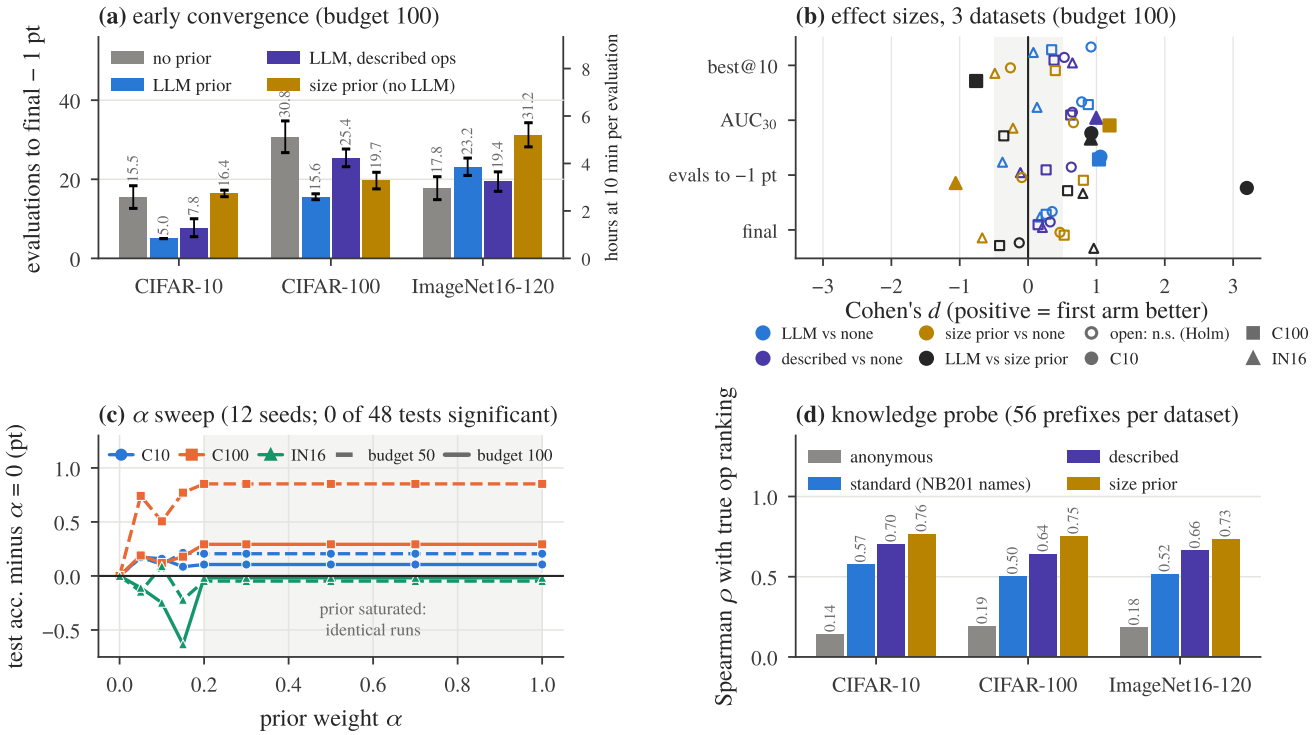


Figure 14. LLM prior. (a) Real evaluations needed to come within one point of the run's final validation accuracy (budget 100, mean ± standard error, 12 seeds); the right axis converts them to hours at 10 minutes per evaluation. The early trajectory under the LLM prior is the same for every seed, so its error bar is near zero. (b) Cohen's d for four paired comparisons on four convergence metrics (best accuracy after 10 evaluations, area under the first 30 evaluations, evaluations to -1 point with the sign flipped so that faster is positive, and final accuracy); filled markers are significant after Holm correction, the grey band marks $|d| < 0.5$. (c) Test accuracy relative to $\alpha = 0$ in a sweep of the prior weight. (d) Spearman correlation between the operation ranking proposed by the prior and the true NAS-Bench-201 ranking over 56 trie prefixes; 0 is chance.

tion of the best achievable accuracy–latency trade-off that its front attains. The fronts are not, however, clearly better than those of the simple rivals (Table 7): all 96 comparisons are ties after Holm correction. Figure 16(a) shows why. ArchTree-HW is ahead of random search from the smallest budget, by 70 to 290 hypervolume units on average, and NSGA-II-style search catches up by budget 200. The largest gain is about 0.3% of the hypervolume. Uncorrected, ArchTree-HW has a higher HV than random search on four of six devices at budget 200, but NSGA-II-style search has a lower IGD on Eyeriss and FPGA. The found fronts are also sparse: only 2–5% of the true front members are found, because 200 proposals sample about 1.3% of the space.

The more useful finding concerns the devices themselves. The true Pareto fronts barely overlap (mean Jaccard index 0.13, at most 0.40 between Eyeriss and FPGA; Figure 16(b)), and only one among the 15,625 architectures lies on the Pareto front of all six devices. Even so, a front that is optimal for one device retains 98% of the attainable hypervolume on another on average (Figure 16(c); worst case 83%, Eyeriss or FPGA to edge GPU). The front transferred between devices also lies close to the true one in inverted generational distance (normalised IGD 0.08 on average, at most 0.38). A front found for one device is therefore a reasonable starting point for another, even though its members are rarely optimal there.

5.11 *Overview of the statistical evidence*

Table 8 collects the 1,404 paired tests of the study. Of the 677 tests in which ArchTree-RMG is compared with another search method (first five rows), 149 are significant wins after Holm correction and one is a loss, so a reader who distrusts any single comparison still faces a consistent direction. The four sensitivity families (UCB setting, α, proxy subsets and hardware; 270 tests) yield only two significant results, which supports the robustness claims above. The LLM families are significant mainly on early-search metrics, never on final accuracy. Friedman tests reject equal performance of all arms in every comparison ($p < 10^{-7}$; Figure 10), and ArchTree-RMG is ranked first on NAS-Bench-201 at equal GPU-hours and within the CD of the best arm elsewhere.

Figure 17 adds the per-run view. Against MCTS-style search, random search, ENAS-style search and LLM-NAS, ArchTree-RMG wins 75–85% of paired runs with confidence intervals well above 0.5. Against RE, ZiCo-style and RZ-style search the probability is close to 0.5, and against LS

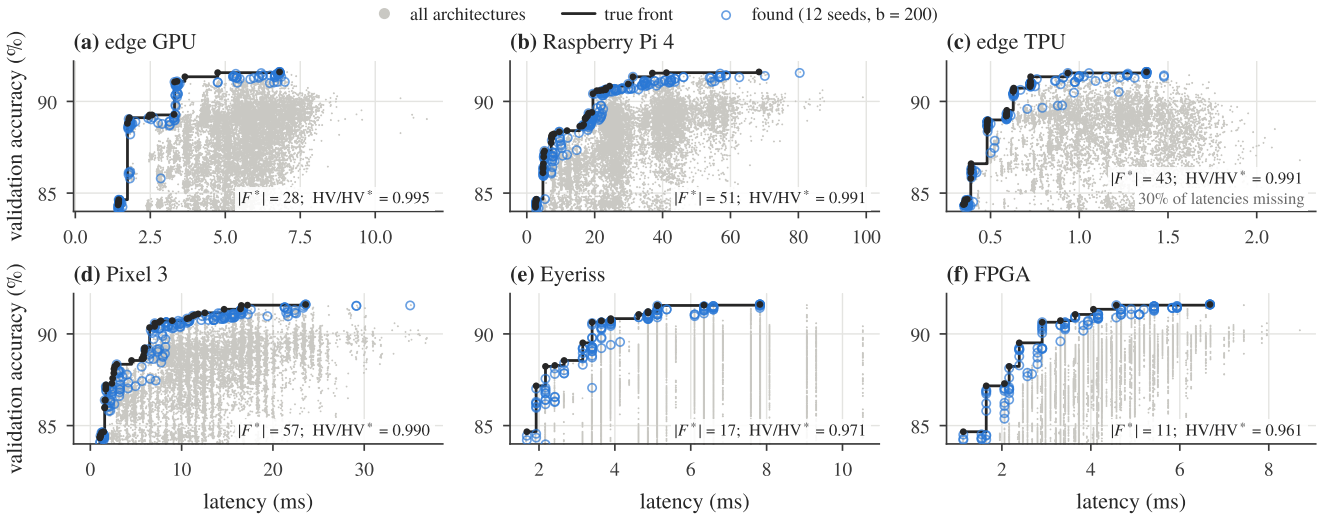


Figure 15. Accuracy–latency trade-off on the six HW-NAS-Bench devices (CIFAR-10, architectures above 84% validation accuracy). Grey: every architecture; black: true Pareto front over all 15,625 architectures; blue circles: members of the fronts found by ArchTree-HW at a budget of 200 (union over 12 seeds). The device determines both the shape of the front and which cells lie on it.

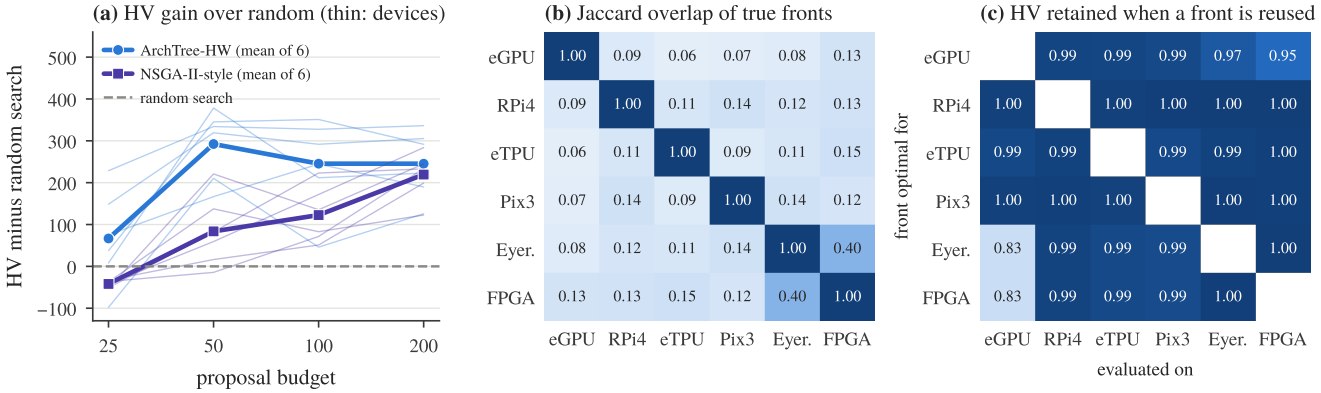


Figure 16. (a) Hypervolume of ArchTree-HW and NSGA-II-style search minus that of random search (thin lines: the six devices; thick: their mean; 12 seeds). (b) Jaccard overlap between the true Pareto fronts of each pair of devices. (c) Share of a device's attainable hypervolume that is retained when the true front of another device is used instead (row: device the front is optimal for; column: device it is evaluated on).

and RE with ZC warm-up it is slightly below (0.43 and 0.41, with intervals that exclude 0.5): these two methods win a few more paired runs at a fixed budget, although never significantly after correction and at about 2.3 times the GPU-hours. At equal GPU-hours the intervals lie above 0.5 on NAS-Bench-201 against every random-search arm, the ungated arms and the $\lambda = 1$ penalty, whereas LS with the gate is the stronger arm on NAS-Bench-101 (0.22) and NATS-Bench SSS (0.38). Equivalence is harder to prove than difference: within ±0.5 points TOST confirms it in 20–25% of the NAS-Bench-201 and NATS-Bench SSS cells and in about three quarters of the NAS-Bench-101 cells, and within ±0.25 points in at most 15%. "No significant difference" between the strong methods therefore means "within a few tenths of a point", not "identical".

6 Discussion

Deduplication is cheap and always worthwhile, yet a hash set provides it as well as a trie does. The label-free gate is the principal source of savings, and its benefit carries over to every proposer we tried. Its bias towards large networks is real but mostly harmless, because accuracy is itself correlated with size; the criterion of Eq. (7) turns this observation into a test that can be run on a sample of architectures before committing a search budget. For a practitioner, the recipe is to estimate r_{PA}, r_{PD} and r_{DA} on a few hundred architectures

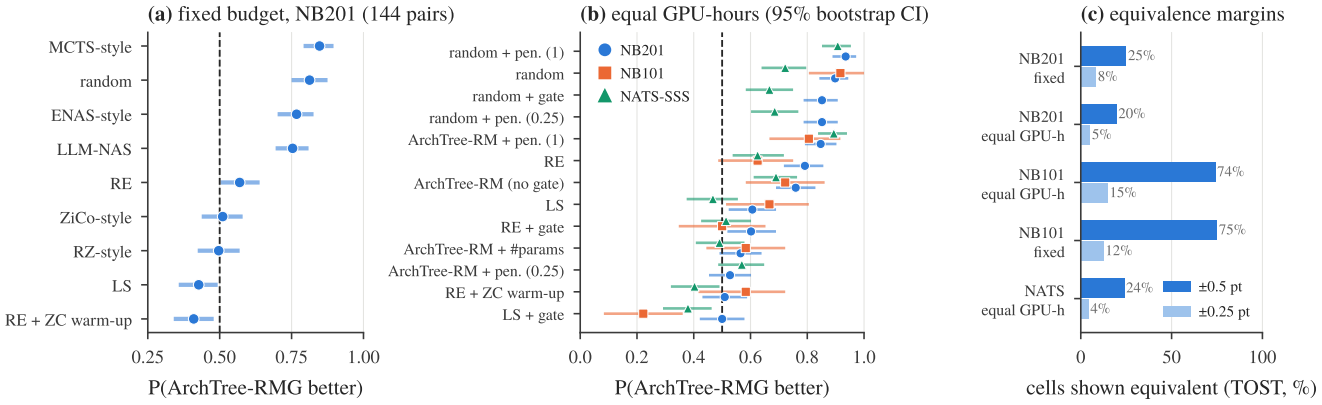


Figure 17. Probability that ArchTree-RMG finds a better architecture than another method on a paired (cell, seed) run, with stratified-bootstrap 95% confidence intervals: (a) against the nine baselines at a fixed proposal budget on NAS-Bench-201; (b) against every arm at equal GPU-hours on the three benchmarks; the dashed line marks 0.5. (c) Share of cells in which two one-sided tests establish equivalence of ArchTree-RMG with the other arms within ±0.5 and ±0.25 points.

Table 8. All paired tests of the study by family: number of tests, tests with $p < 0.05$ before correction, tests significant after Holm correction within the family, and the direction of the significant results.

Family	Tests	Raw	Holm	Significant results
Fixed budget vs 9 baselines	108	38	20	20 RMG wins
Same gate, 6 proposers	240	100	47	47 RMG wins
Equal GPU-hours, 3 benchmarks	261	96	57	57 RMG wins
NB101, same gate	32	16	15	14 wins, 1 loss (RE)
NB101, equal evaluations	36	13	11	11 RMG wins
Held-out design ablation	48	10	4	all favour RMG
Trie vs flat bandit	24	3	1	trie better
Proxy subsets	48	6	2	SVD entropy alone worse
UCB C and backup	78	9	0	–
α sweep	48	11	0	–
LLM convergence (12 metrics)	360	149	109	early-search metrics
LLM knowledge probe	21	21	21	all priors above chance
LLM model size	4	1	0	–
Hardware HV / IGD	96	22	0	–
Total	1,404	495	287	

Raw: $p < 0.05$ without correction. Holm: within the family as defined in section 4 (within baseline, arm or comparison).

(one labelled sample suffices); if $r_{PA}r_{PD} - r_{DA} < 0$, the proxy gate should be used without a cost penalty. A gate on a weak proposer (S2) saves cost but can lose accuracy.

Threats to validity. Internal validity: all methods share one code base, one data store and the same seeds, and test accuracy is never used for any decision; the ArchTree-RMG design was selected using the main seeds, which is why it was re-tested on 12 held-out seeds. Baselines marked "-style" are re-implementations and may differ in detail from their original papers, so their numbers should be read as representative of the family. *Construct validity*: GPU-hours are the training times recorded in the benchmark rather than times measured on our hardware, and the proxy network sees CIFAR-10 images for every dataset. Of the 1,128 equal-GPU-hour runs, five ran out of proposals just before their budget and are kept, flagged. *Statistical conclusion validity*: twelve seeds resolve differences of a few tenths of a point but rarely establish equivalence within ±0.5 points, and many comparisons are corrected for multiplicity, which favours ties; we therefore report raw and corrected outcomes, effect sizes, bootstrap probabilities and TOST side by side. *External validity*: all experiments use tabular benchmarks with image classification on small images; NAS-Bench-101 covers one dataset and was run without the LLM prior or hardware data, and about 30% of edge-TPU latencies are missing from HW-NAS-Bench. Finally, the LLM may have seen NAS-Bench-201 during pre-training; the described-prompt control argues against simple recall but cannot exclude all exposure.

7 Conclusions

We studied a NAS loop that never trains an architecture twice and trains only the proposals that a label-free zero-cost gate ranks in the top 35%. Compared at equal GPU-hours on three benchmarks, it never loses significantly to any of 13 alternatives and stays within the critical difference of the best, while the gate alone halves the training cost of a search at equal accuracy and improves every proposer it is attached to. Zero-cost proxies act as cost detectors in cell-based spaces, though not in a width space, and a simple criterion, $r_{PA}r_{PD} - r_{DA}$, predicts in advance whether correcting a gate for cost will help; on all seven benchmark–dataset pairs it predicted, correctly, that it would not. An LLM prior speeds up the start of the search using general architectural knowledge, but neither the prior nor the tree structure improves final accuracy. Future work should test the gate and the criterion with real training

on larger spaces, add genuine weight inheritance, and extend the LLM prior beyond NAS-Bench-201.

Nomenclature

$\mathcal{A}$	search space of architectures
$a = (o_1, \ldots, o_L)$	architecture as a token sequence of length L
$A(a)$, $T(a)$	validation and test accuracy (%)
$\tau(a)$, $\bar{\tau}$	training time of a; mean training time
H_k	GPU-hour budget, $k\bar{\tau}$
$N(p)$, $n(c)$	visit counts of a parent and a child node
C	UCB exploration constant
r_t	rank-normalised reward
E, S, T	SVD entropy, SynFlow and NASWOT scores
$z(\cdot)$	standardised score
$P(a)$	label-free gate score
q	gate pass rate (0.35)
D	predicted cost, $\log(1 + \#\text{params})$
λ	cost-penalty weight
r_{XY}	normal-score correlation of X and Y
α, α_t	LLM prior weight and its decayed value
τ_K, ρ	Kendall and Spearman rank correlations
CD	Nemenyi critical difference

Appendix A. Derivation of the gate-cost criterion

Let Z_P, Z_D and Z_A be standard normal scores of the proxy, the predicted cost and accuracy, with correlations r_{PD}, r_{PA} and r_{DA}. For $S_\lambda = Z_P - \lambda Z_D$,

$$\mathrm{Cov}(S_\lambda, Z_A) = r_{PA} - \lambda r_{DA}, \tag{A1}$$

$$\mathrm{Var}(S_\lambda) = 1 + \lambda^2 - 2\lambda r_{PD}, \tag{A2}$$

which gives Eq. (6). Writing $g(\lambda) = r_{PA} - \lambda r_{DA}$ and $h(\lambda) = 1 + \lambda^2 - 2\lambda r_{PD}$,

$$\frac{\partial \rho}{\partial \lambda} = \frac{g'(\lambda)}{\sqrt{h(\lambda)}} - \frac{g(\lambda)\, h'(\lambda)}{2\, h(\lambda)^{3/2}}. \tag{A3}$$

At $\lambda = 0$, $g = r_{PA}$, $g' = -r_{DA}$, $h = 1$ and $h' = -2r_{PD}$, so the slope equals $-r_{DA} + r_{PA} r_{PD}$, which is Eq. (7). Because a threshold gate admits the upper tail of S_λ, a higher $\rho(S_\lambda, A)$ means higher expected accuracy among admitted architectures for a fixed pass rate under the Gaussian model.

Acknowledgements

The authors thank the Department of Computer Engineering and Technology, MIT World Peace University, Pune, for computing facilities, and the maintainers of NAS-Bench-101, NAS-Bench-201, NATS-Bench and HW-NAS-Bench for making their benchmarks public.

Declarations

Funding. No external funding was received for this work.
Competing interests. The authors declare no competing interests.
Data and code availability. All benchmarks are public. The code, configuration, seeds, result tables and a reproducibility bundle (environment, package versions and data checksums) are available from the corresponding author on reasonable request and will be released on acceptance.
Use of AI-assisted tools. The open-weight language models gpt-oss-20b and gpt-oss-120b [40] are objects of study in this work (sections 3 and 5.9). Separately, the authors used Claude (Anthropic, Claude Opus 5.5) for coding assistance on the experiment and plotting scripts, for drafting and language editing of the manuscript, and for cross-checking reported numbers against the result files. All results were obtained by running the experiment code on the public benchmarks; no data or results were generated or altered by an AI tool, and every figure was plotted directly from the result files. The authors reviewed all text and analyses, verified every reported number, and take full responsibility for the content.

References

[1] Zoph B and Le Q V 2017 Neural architecture search with reinforcement learning. In: *Proceedings of the International Conference on Learning Representations (ICLR)*

[2] Elsken T, Metzen J H and Hutter F 2019 Neural architecture search: A survey. *J. Mach. Learn. Res.* 20: 1–21

[3] Ren P, Xiao Y, Chang X, Huang P Y, Li Z, Chen X and Wang X 2021 A comprehensive survey of neural architecture search: Challenges and solutions. *ACM Comput. Surv.* 54: 1–34

[4] Real E, Moore S, Selle A, Saxena S, Suematsu Y L, Tan J et al. 2017 Large-scale evolution of image classifiers. In: *Proceedings of the International Conference on Machine Learning (ICML)*, pp. 2902–2911

[5] Pham H, Guan M, Zoph B, Le Q and Dean J 2018 Efficient neural architecture search via parameter sharing. In: *Proceedings of the International Conference on Machine Learning (ICML)*, pp. 4095–4104

[6] Liu H, Simonyan K and Yang Y 2019 DARTS: Differentiable architecture search. In: *Proceedings of the International Conference on Learning Representations (ICLR)*

[7] Ying C, Klein A, Christiansen E, Real E, Murphy K and Hutter F 2019 NAS-Bench-101: Towards reproducible neural architecture search. In: *Proceedings of the International Conference on Machine Learning (ICML)*, pp. 7105–7114

[8] Dong X and Yang Y 2020 NAS-Bench-201: Extending the scope of reproducible neural architecture search. In: *Proceedings of the International Conference on Learning Representations (ICLR)*

[9] Dong X, Liu L, Musial K and Gabrys B 2022 NATS-Bench: Benchmarking NAS algorithms for architecture topology and size. *IEEE Trans. Pattern Anal. Mach. Intell.* 44: 3634–3646

[10] Abdelfattah M S, Mehrotra A, Dudziak Ł and Lane N D 2021 Zero-cost proxies for lightweight NAS. In: *Proceedings of the International Conference on Learning Representations (ICLR)*

[11] Mellor J, Turner J, Storkey A and Crowley E J 2021 Neural architecture search without training. In: *Proceedings of the International Conference on Machine Learning (ICML)*, pp. 7588–7598

[12] Tanaka H, Kunin D, Yamins D L K and Ganguli S 2020 Pruning neural networks without any data by iteratively conserving synaptic flow. *Adv. Neural Inf. Process. Syst.* 33: 6377–6389

[13] Krishnakumar A, White C, Zela A, Tu R, Safari M and Hutter F 2022 NAS-Bench-Suite-Zero: Accelerating research on zero cost proxies. *Adv. Neural Inf. Process. Syst.* 35: 28037–28051

[14] Li C, Yu Z, Fu Y, Zhang Y, Zhao Y, You H et al. 2021 HW-NAS-Bench: Hardware-aware neural architecture search benchmark. In: *Proceedings of the International Conference on Learning Representations (ICLR)*

[15] Real E, Aggarwal A, Huang Y and Le Q V 2019 Regularized evolution for image classifier architecture search. In: *Proceedings of the AAAI Conference on Artificial Intelligence* 33, pp. 4780–4789

[16] White C, Neiswanger W and Savani Y 2021 BANANAS: Bayesian optimization with neural architectures for neural architecture search. In: *Proceedings of the AAAI Conference on Artificial Intelligence* 35, pp. 10293–10301

[17] Li L and Talwalkar A 2020 Random search and reproducibility for neural architecture search. In: *Proceedings of the Conference on Uncertainty in Artificial Intelligence (UAI)*, pp. 367–377

[18] Yang A, Esperança P M and Carlucci F M 2020 NAS evaluation is frustratingly hard. In: *Proceedings of the International Conference on Learning Representations (ICLR)*

[19] White C, Nolen S and Savani Y 2021 Exploring the loss landscape in neural architecture search. In: *Proceedings of the Conference on Uncertainty in Artificial Intelligence (UAI)*, pp. 654–664

[20] White C, Safari M, Sukthanker R, Ru B, Elsken T, Zela A et al. 2023 Neural architecture search: Insights from 1000 papers. arXiv:2301.08727

[21] Wang L, Zhao Y, Jinnai Y, Tian Y and Fonseca R 2020 Neural architecture search using deep neural networks and Monte Carlo tree search. In: *Proceedings of the AAAI Conference on Artificial Intelligence* 34, pp. 9983–9991

[22] Wang L, Xie S, Li T, Fonseca R and Tian Y 2019 Sample-efficient neural architecture search by learning action space. arXiv:1906.06832

[23] Qian G, Zhang X, Li G, Zhao C, Chen Y, Zhang X et al. 2022 When NAS meets trees: An efficient algorithm for neural architecture search. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, pp. 2781–2786

[24] Kocsis L and Szepesvári C 2006 Bandit based Monte-Carlo planning. In: *Proceedings of the European Conference on Machine Learning (ECML)*, pp. 282–293

[25] Bubeck S, Munos R, Stoltz G and Szepesvári C 2011 X-armed bandits. *J. Mach. Learn. Res.* 12: 1655–1695

[26] Munos R 2011 Optimistic optimization of a deterministic function without the knowledge of its smoothness. *Adv. Neural Inf. Process. Syst.* 24: 783–791

[27] Lee N, Ajanthan T and Torr P H S 2019 SNIP: Single-shot network pruning based on connection sensitivity. In: *Proceedings of the International Conference on Learning Representations (ICLR)*

[28] Li G, Yang Y, Bhardwaj K and Marculescu R 2023 ZiCo: Zero-shot NAS via inverse coefficient of variation on gradients. In: *Proceedings of the International Conference on Learning Representations (ICLR)*

[29] Zheng M, Su X, You S, Wang F, Qian C, Xu C et al. 2023 Can GPT-4 perform neural architecture search? arXiv:2304.10970

[30] Chen A, Dohan D M and So D R 2023 EvoPrompting: Language models for code-level neural architecture search. *Adv. Neural Inf. Process. Syst.* 36

[31] Nasir M U, Earle S, Cleghorn C, James S and Togelius J 2024 LLMatic: Neural architecture search via large language models and quality diversity optimization. In: *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO)*. https://doi.org/10.1145/3638529.3654017

[32] Wilcoxon F 1945 Individual comparisons by ranking methods. *Biometrics Bull.* 1: 80–83

[33] Holm S 1979 A simple sequentially rejective multiple test procedure. *Scand. J. Stat.* 6: 65–70

[34] Demšar J 2006 Statistical comparisons of classifiers over multiple data sets. *J. Mach. Learn. Res.* 7: 1–30

[35] Schuirmann D J 1987 A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability. *J. Pharmacokinet. Biopharm.* 15: 657–680

[36] Agarwal R, Schwarzer M, Castro P S, Courville A and Bellemare M G 2021 Deep reinforcement learning at the edge of the statistical precipice. *Adv. Neural Inf. Process. Syst.* 34: 29304–29320

[37] Roy O and Vetterli M 2007 The effective rank: A measure of effective dimensionality. In: *Proceedings of the European Signal Processing Conference (EUSIPCO)*, pp. 606–610

[38] Krizhevsky A 2009 Learning multiple layers of features from tiny images. Technical report, University of Toronto, Toronto

[39] Chrabaszcz P, Loshchilov I and Hutter F 2017 A downsampled variant of ImageNet as an alternative to the CIFAR datasets. arXiv:1707.08819

[40] OpenAI 2025 gpt-oss-120b & gpt-oss-20b model card. arXiv:2508.10925

[41] Deb K, Pratap A, Agarwal S and Meyarivan T 2002 A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Trans. Evol. Comput.* 6: 182–197

[42] Kendall M G 1938 A new measure of rank correlation. *Biometrika* 30: 81–93

[43] Zitzler E and Thiele L 1999 Multiobjective evolutionary algorithms: A comparative case study and the strength Pareto approach. *IEEE Trans. Evol. Comput.* 3: 257–271